

## Documentation-Driven Threat Analysis

# Base Analysis Guide

## Rebuild R7 - Candidate cumulative reconstruction

### Ricostruzione progressiva della guida Base Analysis

Ogni costruito viene ricostruito a partire dall'authority BA corrente e dalle guide precedenti, spiegato in forma generale e soltanto dopo verificato contro esempi aggiornati. Il caso DermaTriage e' evidence applicativa e worked example: non e' la fonte da cui nasce la semantica del costruito.

### RIQUADRO TEMPORANEO - WORKING SET E RIFERIMENTI - DA CANCELLARE PRIMA DELLA GUIDA DEFINITIVA

**Attivi:** DDTA\_DOCUMENTATION\_AUTHORING\_GUIDE\_R7\_REBUILD\_R8.tex; DDTA\_BASE\_ANALYSIS\_GUIDE\_REBUILD\_R7.tex; DDTA\_DERMATRIAGE\_PARALLEL\_CASE\_STUDY\_R25\_FR03\_SPLIT\_REVIEW\_R1.tex.

**Piano corrente:** DDTA\_R25\_BASE\_ANALYSIS\_GUIDE\_REBUILD\_WORK\_PLAN\_R5.md; working analysis: DDTA\_R25\_DERMATRIAGE\_CASE\_STUDY\_WORKING\_ANALYSIS\_R1.md.

**Riferimenti metodologici:** Operational Guide R3 + BA0-BA5 come authority corrente; Core/Complete BA Candidate R1 e precedenti guide operative come research evidence per la ricostruzione didattica.

**Fase corrente:** mantenere R7 cumulativa e ricostruire soltanto i costrutti richiesti dalla BA corrente. transfer, produce, realize, constrain e lo scoped modifier condition restano ricostruiti; questo incremento testa, come candidate non ammessa, la rilocalazione dello storico decisionRule dentro produce come operatorStructure.decisionRule, usando DermaTriage R25 e il regression case Facial Access come pressure.

|                                |                                                                                                                                                                                                                                                                                                                         |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Stato</b>                   | CANDIDATE / NON-NORMATIVE / CUMULATIVE<br>REBUILD                                                                                                                                                                                                                                                                       |
| <b>Repository baseline</b>     | b15990088769e13c7f776a9591b9f0c1d5e11053                                                                                                                                                                                                                                                                                |
| <b>Predecessore cumulativo</b> | DDTA_BASE_ANALYSIS_GUIDE_REBUILD_R6                                                                                                                                                                                                                                                                                     |
| <b>Authority BA corrente</b>   | R3 Operational Guide + BA0 R1 / BA1 R1 / BA2 R3 / BA3 R1 / BA4 R1 / BA5 R1                                                                                                                                                                                                                                              |
| <b>Scopo di R7</b>             | Ricostruzione cumulativa dei costrutti BA richiesti dal caso. I quattro operatori ricostruiti e condition restano invariati; questo incremento testa la candidate operatorStructure.decisionRule, cioe' la rilocalazione dello stesso costruito storico decisionRule dentro produce senza duplicare actor/input/result. |
| <b>Authority change</b>        | Nessuno. decisionRule resta authority corrente come semanticOperatorKey top-level; la sua rilocalazione candidate come operatorStructure.decisionRule richiede regression + checkpoint prima di promotion.                                                                                                              |

# 1 Che cos'è la Base Analysis

La *Base Analysis* è una rappresentazione analitica condivisa del significato di progetto. Il suo compito è rendere esplicite, in una forma comune, le informazioni che devono poter essere riutilizzate, interrogate, confrontate o proiettate verso analisi e viste successive.

Il punto di partenza resta sempre la documentazione. È la documentazione a stabilire che cosa il progetto governa; la Base Analysis organizza quel significato senza sostituirsi alla sua authority. In questo modo consumer diversi possono lavorare sulla stessa base semantica senza dover reinterpretare ogni volta la prosa sorgente.

## Idea fondamentale

La BA rende analiticamente esplicito un significato già governato. Non crea project truth e non diventa una fonte alternativa alla documentazione.

## 1.1 Che cosa resta fuori dalla BA

Una Base Analysis completa non coincide con una copia strutturata di tutto ciò che compare nei documenti. Diagrammi architetturali, DFD, modelli STRIDE e viste di progetto possono essere prodotti a valle, ma non definiscono da soli che cosa debba entrare nella BA. Allo stesso modo, la semplice presenza di un sostantivo, di un verbo, di una tecnologia o di un valore nel testo non basta a giustificare un nuovo elemento analitico.

Il criterio è semantico: rappresentiamo ciò che deve restare distinguibile e riusabile per gli scopi dichiarati, mantenendo il livello di dettaglio effettivamente governato dal progetto.

## 1.2 Perché esiste

Uno stesso significato di progetto può servire a consumer molto diversi. Una vista del sistema può aver bisogno di mostrare responsabilità e interazioni; la traceability deve poter risalire alle fonti; il change impact deve riconoscere che cosa viene toccato da una modifica; test reasoning e threat analysis possono porre domande ancora differenti.

Se ciascun consumer ripartisse direttamente dalla prosa, aumenterebbe il rischio di interpretazioni incompatibili. La BA introduce quindi una base analitica comune: il significato viene ricostruito una volta in modo controllato e poi riutilizzato nelle diverse attività downstream.

## 1.3 Authority, minimalità e feedback

Tre esigenze devono rimanere in equilibrio. La prima è preservare l'autorità documentale: ciò che compare nella BA deve poter essere ricondotto al significato governato. La seconda è la minimalità: aggiungere struttura che non preserva alcuna distinzione utile rende la rappresentazione più pesante senza renderla più corretta. La terza è il feedback: quando una vista o un'analisi incontra un problema, la difficoltà deve poter essere riportata al livello che può davvero risolverla.

Questi tre principi spiegano perché la BA è contemporaneamente una rappresentazione condivisa e un confine: abbastanza ricca da essere riusata, ma non tanto libera da trasformarsi in una seconda documentazione di progetto.

## 1.4 Direzione dell'authority

La governed project documentation rimane la project authority. Una Base Analysis accettata costituisce invece la shared analytical baseline: una rappresentazione comune che può essere consumata da viste, projection, threat method, test model e altri strumenti analitici.

La direzione dell'authority non cambia quando un consumer downstream scopre una difficoltà. Quel consumer può mostrare che una relazione è ambigua, che manca un'informazione necessaria o che la BA non riesce a preservare una distinzione. Il problema torna allora alla BA come domanda di review. Se la BA dispone già del significato necessario, si corregge la rappresentazione; se il significato non è governato, la domanda viene riportata alla documentazione.

### Feedback governato

Una difficoltà downstream può provocare una review della BA e, quando necessario, una domanda documentale. Solo una modifica governata della documentazione può cambiare il project meaning; dopo tale modifica la BA viene rivalidata e le projection interessate vengono ricostruite.

Il feedback migliora quindi la qualità del modello e della documentazione senza invertire la sorgente dell'authority.

## 1.5 Minimum sufficient BA

Una BA piu' dettagliata non e' automaticamente una BA migliore. Il livello giusto e' quello che rende esplicite le distinzioni che devono davvero sopravvivere all'analisi.

Vale la pena introdurre un significato quando serve a mantenere un'identita' riutilizzabile, a distinguere una relazione o una condizione governata, a sostenere traceability e change/revalidation oppure a rispondere a una domanda downstream materiale usando informazioni gia' presenti nella documentazione. Se nessuna di queste esigenze esiste, il dettaglio puo' restare fuori dalla rappresentazione condivisa.

Una domanda downstream puo' quindi motivare l'esplicitazione di un significato gia' governato; non puo' creare il significato che le serve.

### Stopping rule iniziale

Se togliendo un dettaglio non perdiamo una distinzione governata, un'identita' da riusare, una relazione necessaria o la possibilita' di rispondere a una domanda materiale nello scope dichiarato, quel dettaglio non viene aggiunto soltanto per rendere il modello piu' completo graficamente.

### NOT SPECIFIED e documentation gap

Quando la rappresentazione richiederebbe un fatto che la documentazione non governa, il risultato corretto puo' essere NOT SPECIFIED oppure una diagnosis. NOT SPECIFIED non segnala automaticamente un difetto: quel fatto potrebbe semplicemente non appartenere allo scope del progetto. Diventa una documentation question solo quando l'assenza impedisce di esprimere un significato che il progetto dovrebbe governare.

## 2 BA View / Projection Contract

Una stessa Base Analysis deve poter alimentare viste differenti senza trasformare ogni vista in una nuova fonte di significato. Per questo introduciamo un **View Contract**: uno strato esplicito che dichiara quale parte del significato disponibile viene selezionata, come viene proiettata e con quale notazione viene resa visibile.

Quando la sorgente e' la BA, il percorso puo' essere riassunto con un piccolo schema:

Governed Documentation → Accepted Base Analysis → View Contract →  
Rendered View

Non tutte le viste, pero', partono esclusivamente dalla BA. Una vista della gerarchia documentale puo' leggere direttamente MR, Decision e FunctionalRequirement; una vista di traceability puo' invece combinare documentazione e rappresentazione analitica. Il contract deve rendere esplicita questa scelta.

### 2.1 Da quale sorgente parte la view

Una BA\_ONLY view usa soltanto semantica presente nella Base Analysis. Una DOCUMENTATION\_ONLY view rappresenta la struttura o il contenuto governato della documentazione senza duplicarlo artificialmente nella BA. Una DOCUMENTATION\_PLUS\_BA\_TRACEABILITY view mette infine in relazione gli elementi documentali con gli elementi analitici che ne rappresentano il significato.

Questa distinzione evita di inserire nella BA informazioni che servono soltanto a rendere possibile una particolare visualizzazione.

## 2.2 Mermaid come primo target

Mermaid viene adottato come primo target di rendering testuale da investigare. E' utile perche' mantiene i diagrammi come artifact versionabili nel repository e offre piu' famiglie di rappresentazione senza imporre un editor grafico separato.

Mermaid resta pero' una notazione di output. Se la sua sintassi rende comodo introdurre un nodo, una relazione, un participant o un subgraph, quella comodita' non basta a creare nuova semantica nella BA. Prima viene il significato; poi il View Contract decide come renderlo.

### Semantic determinism e no reverse inference

A parita' di Base Analysis accettata e di projection profile, una view canonica MUST esporre lo stesso inventario semantico: stessi referent inclusi, stesse proposition, stessi role/edge meaning, stessi modifier/guard e stessi grouping semanticamente richiesti. Il renderer puo' spostare geometricamente gli oggetti gia' collegati senza violare questo requisito. Una scelta grafica non puo' aggiungere semantica ne' diventare la ragione per modificare la BA.

## 2.3 Che cosa deve dichiarare una view

Il contract deve identificare la view tramite `viewId` e `viewType`; deve poi indicare la sorgente con `sourceType` e `sourceBaseline` e delimitare lo scope. Quando la sorgente comprende la BA, specifica quali `includedReferents` e `includedPropositionsOrLocalStructures` vengono usati. Le trasformazioni sono rese esplicite nelle `projectionRules`; `coverageMode` e `omittedSemantics` spiegano quanto della sorgente viene mostrato, mentre `renderer` e `renderedArtifact` identificano la forma di output.

I campi che non hanno senso per uno specifico `sourceType` vengono marcati come non applicabili, invece di essere riempiti artificialmente.

Una struttura locale presente nella BA, inclusa una futura `operatorStructure`, non deve essere trasformata in uno stato *collapsed* del modello per semplificare il diagramma. Il projection profile puo' ometterne completamente il rendering quando non e' pertinente alla view; `coverageMode/omittedSemantics` rendono esplicita l'omissione senza modificare la BA sottostante.

## 2.4 Coverage mode

BA4 distingue due modi di dichiarare la copertura. Una view EXHAUSTIVE\_FOR\_DECLARED\_SCOPE afferma di rappresentare tutto il significato che appartiene allo scope definito ed e' compatibile con quel tipo di vista. Una view SELECTIVE mostra deliberatamente soltanto una parte del significato disponibile.

La seconda forma e' normale e spesso desiderabile: una vista di responsabilita', per esempio, non deve necessariamente mostrare tutte le informazioni, gli stati o i vincoli presenti nella stessa BA. Cio' che viene omesso da una view selettiva non diventa per questo falso, assente o non governato.

## 2.5 Famiglie di viste da testare

Le prime prove dovranno coprire piu' prospettive. Alcune riguardano la struttura del progetto, come capability, responsabilita' e dipendenze. Altre mostrano aspetti dinamici o relazionali, per esempio information flow, service interaction, state/lifecycle e, quando la semantica disponibile lo consente, pipeline o processi.

Un secondo gruppo riguarda invece la documentazione: gerarchia documentale, traceability fra documentazione e BA, copertura e gap. Non fissiamo ancora il mapping di queste viste; la loro utilita' verra' verificata mentre costruiamo la BA di DermaTriage.

Il fatto che una famiglia di viste sia utile non dimostra che la BA corrente contenga gia' tutto cio' che serve a produrla.

## 2.6 Quando una view non e' producibile

Una view puo' mettere in evidenza un'assenza senza avere il diritto di colmarla. Se il significato richiesto non e' presente nella BA, il View Contract registra `VIEW REQUIRES MISSING BA SEMANTICS`. Se, a sua volta, la BA non puo' ricavare quel significato dalla documentazione governata, il problema diventa `BA REQUIRES MISSING GOVERNED MEANING`.

Questa distinzione permette di riportare la difficolta' al livello corretto invece di risolverla con una convenzione grafica o una supposizione del renderer.

## 2.7 Come una relazione entra nella view

Ogni elemento visuale che porta significato deve poter essere giustificato da una regola di projection dichiarata. Un edge Mermaid, per esempio, non e' valido perche' esiste una freccia con un nome simile nella notazione: deve essere il risultato di una trasformazione esplicita di semantica documentale o BA.

La notazione decide come mostrare il risultato della projection; non decide che cosa quel risultato significa. Per questo i mapping fra DDTA, BA e renderer verranno definiti e testati separatamente, invece di essere dedotti per somiglianza terminologica.

### 3 Dal testo documentale alla Base Analysis

La Base Analysis non nasce traducendo meccanicamente le frasi della documentazione in nodi e relazioni. Prima di scegliere un elemento BA dobbiamo capire quale significato il testo stabilisce realmente e quali parti di quel significato devono rimanere distinguibili nell'analisi.

La forma linguistica e la struttura analitica non coincidono necessariamente. Una frase può contenere più fatti rilevanti; più frasi possono contribuire allo stesso significato; una parola tecnica può descrivere una realizzazione senza richiedere una nuova identità analitica.

#### Prima il significato, poi il costruito

La domanda iniziale non è quale operatore somigli al verbo che stiamo leggendo. È quale significato di progetto resti vero indipendentemente da come la frase è stata scritta.

Da qui si procede per gradi. Si parte dalla documentazione governata, si ricostruisce il significato che essa effettivamente stabilisce, si isolano i fatti semantici minimi che servono e si chiarisce quali identità e quali asserzioni hanno bisogno di una forma analitica. Solo allora si materializzano BAReferent e BAProposition e, in un passaggio successivo, si sceglie il costruito formale più adatto.

#### Nessun word-to-model mapping

La presenza di un sostantivo non crea automaticamente un BAReferent; un verbo non seleziona da solo un operatore; il confine di una frase non determina il confine di una BAProposition.

### 3.1 Governed semantic fact

Chiamiamo *governed semantic fact* il piu' piccolo significato governato che e' utile isolare per decidere come rappresentarlo nella BA. Non e' un nuovo tipo di elemento BA: e' un passaggio di analisi che ci permette di separare la comprensione del testo dalla scelta della sintassi analitica.

Un fatto candidato deve sempre poter essere giustificato tornando alla documentazione. Se per formularlo dobbiamo aggiungere un attore, un valore, un ordine, una causalita', una responsabilita' o una condizione non stabilita dal progetto, non stiamo piu' isolando il significato: lo stiamo completando.

#### DermaTriage - dominio della priorita'

La documentazione stabilisce che, quando DermaTriage produce una priorita' operativa di triage, il valore appartiene all'insieme P1, P2, P3, P4. Possiamo quindi isolare il fatto che **Operational-Priority ha come dominio ammesso P1-P4**. In questo momento non serve ancora decidere se OperationalPriority debba essere un BAREferent o quale operatore rappresentera' il vincolo.

#### DermaTriage - percorso senza immagine

Nel percorso symptom-only emergono almeno due significati distinguibili: l'assenza dell'immagine non interrompe il triage e l'urgenza viene determinata usando le informazioni sintomatologiche disponibili sullo stesso caso. Tenerli separati evita di trasformare immediatamente l'intero paragrafo in una sola asserzione sovraccarica.

### 3.2 Come isolare i fatti senza inventarli

La segmentazione serve a preservare differenze che potrebbero avere identità, relazioni, condizioni o provenance diverse; non serve a spezzare il testo fino a ottenere una proposition per ogni frase o per ogni verbo.

Il lavoro parte dal commitment governato e separa soltanto le distinzioni che potrebbero cambiare o essere riutilizzate indipendentemente. Le parti che perderebbero significato se isolate restano unite. Se il testo ammette più letture materialmente diverse, l'ambiguità rimane visibile: non viene risolta usando conoscenza di dominio, convenzioni tecniche o dettagli dell'implementazione.

#### Delete test sul fatto candidato

Se elimino questa distinzione prima di costruire la BA, perdo qualcosa che la documentazione governa e che potrebbe diventare rilevante per reuse, traceability, change o una domanda downstream? Se la risposta è no, probabilmente non serve isolarla come fatto autonomo.

### 3.3 Il livello documentale resta visibile

MR, Decision e FunctionalRequirement possono governare la stessa area con livelli di precisione diversi. La BA non appiattisce questa progressione. Un MR può stabilire un risultato e i suoi input in termini generali; una Decision può restringere una strategia; un FR può precisare un dominio, una condizione o una regola operativa.

Quando la documentazione introduce un dettaglio più specifico, la BA può aggiungere la relativa asserzione senza cancellare automaticamente il significato più generale che rimane valido.

## 4 Gli elementi fondamentali della Base Analysis

La Base Analysis distingue due famiglie fondamentali di elementi: `BAReferent` e `BAProposition`. La prima serve a dare identità analitica a un significato di progetto; la seconda a dare identità a un'asserzione analitica su uno o più significati.

BAE può essere usato come termine ombrello per riferirsi agli elementi della Base Analysis, ma non introduce una terza famiglia.

### Due domande diverse

Quando incontriamo un significato chiediamo se deve poter essere riconosciuto e riutilizzato come identità analitica. Quando incontriamo un'asserzione chiediamo invece se deve poter essere tracciata, revisionata o diagnosticata indipendentemente. La prima domanda porta a un `BAReferent`; la seconda a una `BAProposition`.

Questa distinzione è semantica, non implementativa. Un tool può memorizzare referent e proposition in classi, tabelle o grafi differenti; ciò che deve restare stabile è la differenza tra la cosa cui attribuiamo identità e l'asserzione che facciamo su di essa.

## 4.1 BAReferent: dare identita' a un significato

Un BAReferent e' un'unita' di significato di progetto che deve poter essere riconosciuta e riutilizzata indipendentemente nell'analisi. Non e' limitato agli oggetti software o ai componenti runtime: puo' rappresentare una capability, un'informazione, un risultato, un comportamento, un evento, uno stato, un contratto, uno store, un boundary o un altro significato che abbia bisogno di identita' stabile.

### Identity test

La domanda utile non e' "questo nome compare nel testo?", ma "questo significato deve vivere oltre l'asserzione in cui l'ho incontrato?". L'identita' separata diventa giustificata quando il significato deve essere riutilizzato in piu' proposition, qualificato o vincolato, correlato o referenziato, confrontato fra baseline, seguito attraverso change/revalidation oppure interrogato come oggetto distinto da un consumer downstream.

Se nessuna di queste esigenze esiste e la rimozione dell'identita' non fa perdere significato governato, un BAReferent separato puo' essere superfluo.

La presenza lessicale, da sola, non basta: tecnologie, valori, campi e sostantivi diventano referent solo quando serve davvero un'identita' analitica indipendente.

## 4.2 BAReferent: prima applicazione a DermaTriage

La documentazione consolidata di MR-01 offre già alcuni buoni esempi per applicare l'identity test senza congelare prematuramente il registry finale.

**DermaTriage.** Compare ripetutamente come soggetto di obblighi e relazioni; e' quindi un forte candidato a identita' BA riusabile.

**Caso dermatologico.** Collega immagine, sintomi e risultati. La necessita' di una sua identita' autonoma dipendera' soprattutto da come dovremo preservare correlation e same-case semantics nelle proposition successive.

**Urgenza e priorita' operativa.** Sono risultati governati distinti. L'urgenza viene prodotta e riutilizzata nei passaggi successivi; la priorita' e' ulteriormente vincolata alla P-scale. Entrambe sono candidate forti a identita' riusabile, mentre resta da decidere il livello corretto fra concetto e occurrence per singolo caso.

**Il valore P1.** Il fatto che sia governato non lo trasforma automaticamente in un BAReferent. Se non deve essere riutilizzato come identita' indipendente, puo' rimanere un valore locale controllato.

### L'esempio non decide ancora il modello finale

Questa prima applicazione mostra come usare l'identity test. Non congela il registry DermaTriage e non decide ancora la granularita' instance-vs-concept: saranno le proposition necessarie a mostrare quali identita' devono sopravvivere autonomamente.

Differenze come actor, component, information, behavior o state non richiedono di per se' famiglie BA diverse. Finche' classificazioni, ruoli, valori e proposition riescono a preservarle onestamente, restano all'interno della stessa famiglia BAReferent.

### 4.3 BAProposition: dare identita' a una asserzione

Una `BAProposition` e' un'asserzione analitica methodology-neutral su uno o piu' `BAReferent`. La sua identita' permette di collegare provenance, review, diagnosis e change disposition all'asserzione stessa, senza confonderla con i significati che mette in relazione.

Questa distinzione diventa importante quando un referent rimane stabile ma cambia cio' che il progetto afferma su di esso. In quel caso non dobbiamo sostituire l'identita' del referent: possiamo introdurre, modificare, supersedere o diagnosticare le proposition interessate.

A livello BA2 la forma generale corrente prevede un `semanticOperatorKey`, una o piu' `participation` con `roleKey` scoped all'operatore e, quando ammessi, `polarity`, `modifier` e strutture locali. Le firme specifiche verranno spiegate piu' avanti costruito per costruito.

```
BAProposition
  semanticOperatorKey
  participation [1..*]
    roleKey
    term
  polarity
  scopedModifier [0..*]
  operatorStructure [0..1, when admitted]
```

Il `term` di una `participation` e' normalmente un `BAReferent`. Un valore locale controllato e' sufficiente soltanto quando quel significato non richiede identita' indipendente.

#### 4.4 BAReferent e BAProposition lavorano insieme

Prendiamo il FunctionalRequirement che governa la priorit  operativa P1-P4. Prima di formalizzare, riconosciamo due cose diverse. Da un lato esistono significati che devono poter essere riutilizzati, come DermaTriage e OperationalPriority; dall'altro la documentazione afferma qualcosa su questi significati, per esempio che una priorit  operativa puo' essere prodotta e che il suo dominio e' limitato a P1-P4.

L'identity test suggerisce quindi almeno i referent candidati DermaTriage e OperationalPriority. Le affermazioni sul risultato e sul dominio saranno invece materializzate come BAProposition quando avremo selezionato operatori, ruoli e valori compatibili con il significato governato.

##### Perche' non formalizziamo troppo presto

Possiamo gia' vedere che disponibilit  della priorit  e dominio ammesso sono due asserzioni distinguibili. Non serve pero' scegliere adesso produce, constrain o un altro costrutto: prima fissiamo la grammatica generale, poi riesaminiamo ogni operatore con lo stesso contratto descrittivo.

Il punto da conservare e' semplice: il referent rappresenta il significato che deve continuare a esistere come identita' analitica; la proposition rappresenta cio' che affermiamo su quel significato. Un grafo o un renderer non devono confondere i due livelli soltanto per ottenere una forma visuale piu' comoda.

## 5 Granularita' e costruzione della Base Analysis

La granularita' corretta non dipende dalla lunghezza della frase ne' dal numero di elementi che un tool riesce a disegnare. Dipende da quali distinzioni devono poter essere identificate, verificate e cambiate indipendentemente.

### 5.1 Quando separare due proposition

Due significati meritano proposition separate quando possono divergere per provenance o stato di review, per condizione di applicabilita', per impatto di un cambiamento, perche' uno dei due diventa target di una relazione successiva oppure perche' consumer downstream diversi hanno bisogno di distinguerli. Se invece le parti hanno senso soltanto insieme, la rappresentazione deve conservarne l'unita'.

Separare significa quindi preservare indipendenza semantica, non frammentare il testo.

### 5.2 Procedura iniziale di costruzione

Il lavoro puo' essere condotto in sette passaggi:

1. fissare baseline, authority e scope documentale;
2. leggere il testo source-first e ricostruire il significato governato prima di pensare alla sintassi BA;
3. isolare soltanto i governed semantic fact che devono rimanere distinguibili;
4. applicare l'identity test e riconoscere le asserzioni che richiedono BAREferent e BAProposition;
5. verificare che la cardinalita' BA derivi dal significato analitico e non dalla forma dell'albero documentale; se emerge una possibile inefficienza strutturale, registrare un finding e rinviare la decisione al layer documentale;
6. selezionare operatori, ruoli, valori e strutture locali solo dopo che identita' e asserzioni sono chiare;
7. tornare alle fonti per verificare ogni elemento, preservare NOT SPECIFIED o diagnosis quando necessario e applicare infine stopping rule e forbidden-inference review.

#### Stop prima degli operatori

Prima di entrare nel catalogo dei costrutti, il lettore deve saper spiegare perche' un significato richiede un BAREferent, perche' un'asserzione richiede una BAProposition e quali parti della documentazione giustificano entrambi. Se questo non e' chiaro, scegliere un operatore e' prematuro.

## 6 Struttura documentale, cardinalita' BA e feedback verso monte

La documentazione DDTA organizza responsabilita', decisioni e obblighi in una struttura utile per authoring, implementazione, verifica e change management. La Base Analysis ha uno scopo diverso: rendere espliciti i significati analitici che devono essere riutati e interrogati. Per questo le due strutture non hanno una cardinalita' obbligatoriamente coincidente.

### Regola di indipendenza strutturale

Gli elementi documentali sono sorgenti di significato governato, non un template da tradurre uno-a-uno in elementi BA.

```
numero di branch documentali
    !=
numero di BAReferent
    !=
numero di BAProposition
```

Piu' elementi documentali possono sostenere uno stesso significato analitico; un singolo elemento documentale puo' invece contenere piu' asserzioni che devono restare distinguibili in BA. La cardinalita' corretta emerge dai test di identita', asserzione, provenance, change e consumo downstream, non dal numero di nodi presenti nell'albero DDTA.

### Traduzione meccanica vietata

Non creare un BAReferent, una BAProposition o un futuro ramo analitico soltanto perche' esiste un elemento documentale distinto. Allo stesso modo, non comprimere due significati BA distinti soltanto perche' sono scritti nello stesso requisito.

## 6.1 Quando la BA espone una pressione strutturale documentale

Durante la normalizzazione puo' emergere che un branch documentale contiene un significato reale e source-supported ma non possiede conseguenze downstream sufficientemente autonome da giustificare la propria esistenza come branch separato. Questo e' un risultato utile della BA, ma non autorizza la BA a riscrivere la documentazione.

### Finding di review strutturale

Usare POSSIBLE\_UPSTREAM\_STRUCTURAL\_REDUNDANCY come segnale di review quando il significato di un branch puo' essere preservato senza perdita dentro il branch che possiede il comportamento correlato e la separazione autonoma non mostra una utilita' downstream distinguibile.

Il finding deve registrare almeno:

- gli elementi documentali coinvolti;
- il significato governato che ciascuno porta;
- le distinzioni analitiche che devono comunque essere preservate;
- le conseguenze downstream autonome cercate;
- perche' la separazione appare necessaria oppure potenzialmente superflua;
- la review documentale da riaprire.

### No authority inversion

La BA puo' diagnosticare la pressione e mantenere una cardinalita' analitica non artificiale. Non puo' dichiarare da sola MERGE, cancellare un elemento documentale o trasferirne l'ownership. La decisione strutturale torna alla Documentation Authoring review; se la documentazione cambia, la BA viene poi revalidata o ricostruita.

## 6.2 Pressure case DermaTriage: dominio P1-P4 e derivazione della priorit 

La ricostruzione DermaTriage ha mostrato il caso in forma concreta. In una revisione precedente il significato relativo alla priorit  operativa era separato in due branch:

```
DEC-MR01-02
  adozione del dominio P1-P4
  -> FR-MR01-02-01
    la priorit  appartiene a {P1,P2,P3,P4}

DEC-MR01-04
  separazione urgenza analitica / priorit  operativa
  -> FR-MR01-04-01
    regola che deriva P1-P4 da urgenza e confidence
```

La review ha chiarito che il significato “la priorit  operativa usa il dominio P1-P4”   reale e deve essere preservato. Non era quindi corretto descrivere DEC-MR01-02 come priva di semantica. Il problema era diverso: come branch autonomo non produceva un comportamento, una verifica, una modifica indipendente o una analisi distinta rispetto al branch che governa la derivazione della priorit .

### Esito documentale del pressure case

Il dominio P1-P4 viene mantenuto dentro DEC-MR01-04; FR-MR01-04-01 continua a governare il mapping operativo. Il branch DEC-MR01-02 / FR-MR01-02-01 puo’ quindi essere consolidato senza perdere il significato del dominio.

Questo caso mostra due regole contemporaneamente:

1. una distinzione semantica puo’ restare necessaria nella BA anche dopo la riduzione del numero di branch documentali;
2. il numero di distinzioni BA non dimostra quanti branch debbano esistere nella documentazione.

### 6.3 Ritorno alla Documentation Authoring review

La decisione sulla struttura documentale usa il **Downstream Utility Test** definito nella Documentation Authoring Guide:

#### Downstream Utility Test - eseguito upstream

Se elimino questo branch come branch autonomo e trasferisco il suo significato nella Decision che possiede i comportamenti correlati, perdo una distinzione necessaria per implementazione, verifica, modifica indipendente o analisi?

Se la risposta e' positiva, la separazione puo' essere giustificata. Se e' negativa, il layer documentale riesamina il branch per MERGE o REWORK. In entrambi i casi la BA non anticipa l'esito.

```
governed / working documentation
-> source-first BA normalization
-> structural-feedback finding
-> Documentation Authoring review
-> Downstream Utility Test
-> KEEP / MERGE / REWORK documentale
-> BA revalidation if documentation changed
```

#### Distinzione da preservare

“Due proposition analitiche” e “due branch documentali” sono domande diverse. La BA separa i significati quando serve all'analisi; la documentazione separa i branch quando serve al lavoro sul progetto. Le due cardinalita' possono coincidere, ma non sono obbligate a farlo.

#### Confine di questa revisione

La R7 mantiene il catalogo in forma cumulativa: transfer, produce, realize, constrain e condition sono gia' ricostruiti. Il nuovo pressure case non autorizza a promuovere automaticamente un quinto operator: la rebuild testa invece se lo storico decisionRule possa diventare operatorStructure.decisionRule della proposition produce. Fino a regression e checkpoint, decisionRule resta authority corrente e la rilocalazione resta candidate non ammessa.

## 7 Operatore transfer

CURRENT / PRESERVED

### 7.1 Che cosa rappresenta

`transfer` rappresenta la **conveyance governata** di uno o piu' contenuti da una `source` identificabile verso una o piu' `destination` identificabili. Il costrutto non descrive genericamente un collegamento, un ordine di esecuzione o una vicinanza architetturale: il suo **smallest semantic fact** e' che *un content viene trasferito da A a B*.

#### Smallest semantic fact

Esiste un contenuto governato X la cui conveyance e' asserita da una `source` A verso almeno una `destination` B.

### 7.2 Quando usarlo

Usare `transfer` quando la documentazione governa sufficientemente tutti e tre i ruoli obbligatori: origine, destinazione e contenuto trasferito. Il verbo concreto della documentazione puo' essere "consegnare", "inviare", "rendere disponibile a", "passare", "trasmettere" o un'espressione equivalente; la scelta non dipende dalla parola usata ma dal significato governato.

### 7.3 Quando non usarlo

Non accettare una `proposition transfer` se almeno uno fra `source`, `destination` e `content` deve essere inventato per completare la frase. La presenza di due componenti nello stesso diagramma, di una chiamata API, di una sequenza di stage o di un dato disponibile in memoria non basta da sola a dimostrare conveyance.

#### Gate minimo

Se la `source` non permette di identificare `source`, almeno una `destination` e almeno un `content`, il risultato corretto e' INSUFFICIENT EVIDENCE / NOT SPECIFIED, non un `transfer` completato per plausibilita'.

## 7.4 Firma canonica

La firma corrente preservata da BA2 R3 e':

```
transfer
  behavior    -> BAREferent [0..1]
  source      -> BAREferent [1]
  destination -> BAREferent [1..*]
  content     -> BAREferent [1..*]
```

polarity appartiene alla BAProposition e resta obbligatoria secondo il contratto generale. Gli scoped modifier condition e temporalScope compaiono soltanto quando la source limita realmente l'asserzione.

| Role        | Card. | Significato                                                                                                                                                                         |
|-------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| behavior    | 0..1  | Identita' riusabile dello specifico comportamento di transfer, solo quando il segmento deve essere target di altre proposition, vincoli, realization, confronto o analisi autonoma. |
| source      | 1     | Origine governata della conveyance.                                                                                                                                                 |
| destination | 1..*  | Uno o piu' destinatari governati del content.                                                                                                                                       |
| content     | 1..*  | Significato informativo/materiale la cui conveyance e' governata.                                                                                                                   |

## 7.5 Identita' opzionale del segmento

behavior non viene creato per ogni scambio. Riceve identita' BAREferent quando il trasferimento stesso deve poter essere riusato, vincolato, realizzato, protetto, confrontato fra baseline o preso come target da piu' asserzioni.

### Behavior identity test

Se elimino l'identita' dello specifico segmento di transfer, perdo la possibilita' di esprimere un significato governato che riguarda proprio il passaggio e non soltanto source, destination o content?  
Se no, behavior resta assente.

## 7.6 Continuità di identità del content

Quando un significato viene prodotto, trasferito e poi usato da un consumer senza cambiare identità semantica, la BA riassume lo stesso BAREferent. La posizione “output”, “content del transfer” e “input” non crea tre identità diverse.

```
producer result    -> X
transfer content   -> X
consumer input     -> X
```

### Identity continuity rule

Un nuovo BAREferent è giustificato solo quando tra i due punti compare una trasformazione che stabilisce un significato semanticamente distinto. Cambio di variabile, serializzazione, formato fisico o collocazione non bastano da soli.

## 7.7 Invarianti

- ogni transfer accettato possiede esattamente una source e almeno una destination e un content;
- i roleKey sono operator-scoped: source significa origine della conveyance, non una classificazione universale del referent;
- destination multiple e content multipli sono ammessi solo quando la stessa asserzione governata li comprende;
- l'eventuale behavior identifica il segmento, non sostituisce source, destination o content;
- polarity negativa nega l'asserzione di transfer soltanto quando questa negazione è realmente governata; assenza di evidence non equivale a transfer negativo.

## 7.8 Forbidden inference

Dal solo transfer non inferire protocollo, sincronicità, cifratura, affidabilità, successo della consegna, persistenza, ownership, storage, service consumption, transport medium, retries, acknowledgement, failure semantics o topologia intermedia. Questi significati richiedono evidence e costrutti appropriati separati.

## 7.9 Domande da porre alla documentazione

Le domande servono a restringere l'interpretazione; non sono un classificatore automatico.

1. La source governa realmente che un contenuto passi da un'origine a un destinatario?
2. Qual e' la source? E' esplicitamente identificabile senza inferenza architetturale?
3. Qual e' o quali sono le destination?
4. Qual e' o quali sono i content trasferiti?
5. Source, destination e content hanno gia' un BAREferent con la stessa identita' semantica? Se si', riusarlo.
6. Il content mantiene la stessa identita' fra producer, transfer e consumer oppure la source governa una trasformazione semanticamente distinta?
7. Il segmento di transfer deve essere target autonomo di property, constraint, realization, security scope, change impact o altre proposition? Se si', valutare behavior.
8. La proposition vale sempre oppure solo sotto una condition o un temporal scope governato?
9. La source afferma il transfer o lo nega esplicitamente? Non dedurre polarity negativa dall'assenza di trasferimento.
10. Rimane qualche ruolo obbligatorio non stabilito? Se si', fermarsi: il dato resta NOT SPECIFIED oppure diagnosis.

### Procedura minima

Leggere il smallest governed semantic fact; identificare o riusare i BAREferent necessari; verificare source/destination/content; decidere se serve behavior; compilare una sola BAProposition minima; aggiungere scoped modifier solo se governati; infine controllare le forbidden inference.

## 7.10 Esempio generico minimo

### Documentazione inventata ma coerente

“Il sottosistema di acquisizione MUST consegnare il MeasurementRecord al motore di valutazione.”

Assumendo che le tre identità abbiano superato il normale gate BA1, la proposition minima è:

```
transfer
  source      -> MeasurementAcquisition
  destination -> EvaluationEngine
  content     -> MeasurementRecord
  polarity    -> affirmative
```

Non serve un behavior referent se il segmento non deve essere riusato o target di altre proposition.

## 7.11 Esempio corrente - DermaTriage T06

Nella documentazione corrente, Stage 2 produce una ClinicalDescription e la rende disponibile allo Stage 3; Stage 3 usa quella stessa descrizione come input del retrieval. L'estrazione corrente è:

```
transfer
  behavior     -> ClinicalDescriptionTransfer
  source       -> ClinicalDescriptionGeneration
  destination  -> HistoricalCaseRetrieval
  content      -> ClinicalDescription
  polarity     -> affirmative
```

ClinicalDescription è lo stesso BAREferent già usato come result della generazione e come input del retrieval. ClinicalDescriptionTransfer riceve identità perché il segmento è oggetto autonomo della review data-path / information-contract; questa scelta appartiene all'esempio, non alla definizione universale di transfer.

### Boundary example

“Stage B usa X” non basta, da solo, a costruire transfer se la documentazione non stabilisce da quale source X arrivi. L'input può essere noto mentre il transfer resta INSUFFICIENT EVIDENCE.

## 7.12 Prompt LLM di supporto all'autoring

Il prompt e' un ausilio operativo: non cambia la semantica BA e non rende autorevole una inferenza non supportata. Deve ricevere l'estratto documentale governato e, quando disponibile, il registro dei BAReferent gia' esistenti.

TASK: identify governed BA transfer propositions.

Use ONLY the supplied documentation and existing BA registry.

A transfer exists only if the source supports:

source [1], destination [1..\*], content [1..\*].

Rules:

1. Identify the smallest governed conveyance fact.
2. Reuse existing BAReferents when semantic identity is unchanged.
3. Propose a new BAReferent only when BA1 independent identity is needed.
4. Add behavior only if the transfer segment itself must be independently targeted, constrained, realized, compared, traced or analyzed.
5. Preserve the same content BAReferent across producer/transfer/consumer when the governed meaning is unchanged.
6. Use polarity=negative only for an explicitly negated transfer assertion.
7. Add condition/temporalScope only when explicitly governed.
8. If source, destination or content is missing, return INSUFFICIENT\_EVIDENCE.
9. Do not infer protocol, network topology, persistence, success, ownership, encryption, service consumption, transport medium or failure semantics.
10. Do not invent operators, roles, states or metadata.

Return EXACTLY the authoring wrapper and BA output shape shown next.

### Human review remains mandatory

L'LLM puo' proporre referent e proposition; l'accettazione richiede comunque verifica contro source, identity criterion, canonical naming e provenance. Una risposta sintatticamente completa non sana un gap documentale.

### 7.13 Output esatto atteso dall'estrazione assistita

Il wrapper seguente e' un formato di authoring/review, non un nuovo costrutto BA. La parte di output BA usa soltanto la shape ammessa.

```
TRANSFER_ASSESSMENT
  decision = ACCEPT_TRANSFER | NOT_A_TRANSFER | INSUFFICIENT_EVIDENCE
  evidence = <source locator / concise supported wording>
  notSpecified = [<only unresolved material items>]

BA_OUTPUT
  BAREferent reuse/candidates:
    <canonical names only as needed>

  BAProposition <candidate-id>
    semanticOperatorKey = transfer
    behavior    -> <BAREferent>      # omit line if not justified
    source      -> <BAREferent>
    destination -> <BAREferent>      # repeat for 1..*
    content     -> <BAREferent>      # repeat for 1..*
    polarity    = affirmative | negative
    scopedModifier = <condition|temporalScope> # omit if absent
```

Se decision != ACCEPT\_TRANSFER, BA\_OUTPUT non contiene una proposition transfer accettabile. I nomi proposti restano candidate finche' non superano canonical naming, provenance e review.

### 7.14 Discriminazioni con altri costrutti

-

## 7.15 Rappresentazione Mermaid di transfer

Per transfer, la projection rende direttamente leggibili i role dell'operatore. `source` e `destination` sono gli endpoint; `content` occupa la posizione centrale ed e' mostrato con una forma documento. L'intero percorso direzionale `source -> content -> destination` rappresenta **una sola** `BAProposition transfer`: le due frecce sono una decomposizione grafica del percorso, non due proposition.

Quando il role opzionale `behavior` e' presente, viene visualizzato come annotazione distinta del transfer: nodo giallo collegato al `content` con una linea tratteggiata e non direzionale. Questa linea e' una convenzione di projection del role `behavior`; non rappresenta una seconda relazione BA.

| Parte di transfer         | Rappresentazione Mermaid                                                                             |
|---------------------------|------------------------------------------------------------------------------------------------------|
| <code>source</code>       | Nodo endpoint all'origine del percorso.                                                              |
| <code>content</code>      | Nodo centrale con forma documento e sfondo bianco.                                                   |
| <code>destination</code>  | Nodo endpoint raggiunto dal percorso direzionale.                                                    |
| <code>behavior</code>     | Nodo opzionale con sfondo giallo; linea tratteggiata non direzionale verso il <code>content</code> . |
| Proposition id / polarity | Metadati della stessa <code>BAProposition</code> ; non diventano un nodo del data path.              |

```

flowchart LR
  S["ClinicalDescriptionGeneration<br/>source"]
  C@{ shape: doc, label: "ClinicalDescription<br/>content" }
  D["HistoricalCaseRetrieval<br/>destination"]
  B["ClinicalDescriptionTransfer<br/>behavior"]

  S --> C --> D
  B -.- C

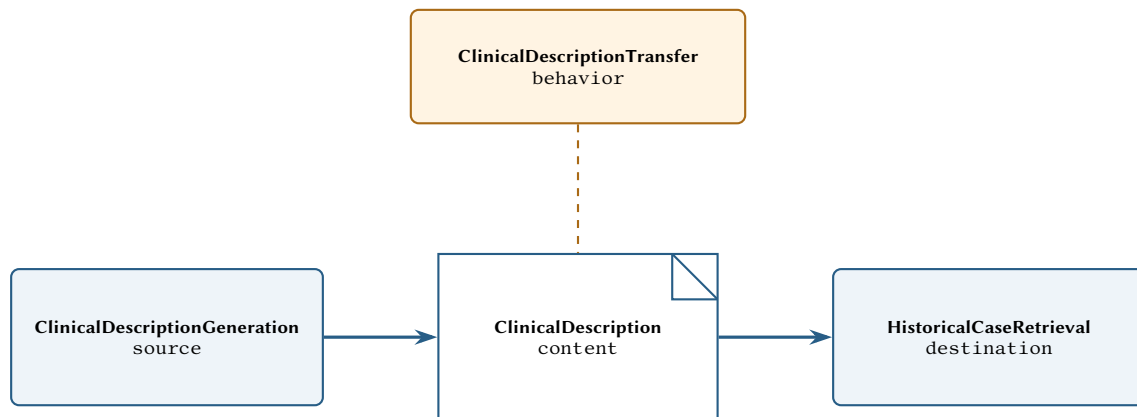
  classDef endpoint fill:#EEF4F9,stroke:#275D86,color:#17324D
  classDef content fill:#FFFFFF,stroke:#275D86,color:#17324D
  classDef behavior fill:#FFF4E3,stroke:#A96A16,color:#17324D
  class S,D endpoint
  class C content
  class B behavior
  linkStyle 2 stroke:#A96A16,stroke-width:1.5px,stroke-dasharray:5 5

```

Il codice usa il caso T06 come esempio concreto. Se `behavior` fosse assente, il nodo giallo e la linea tratteggiata verrebbero omessi; il percorso `source -> content -> destination` resterebbe invariato.

## 7.16 Esempio grafico corrispondente

Il diagramma seguente applica la stessa convenzione al caso T06: il documento bianco e' il content; il percorso con frecce rende la direzione del t transfer; il nodo giallo identifica il behavior opzionale senza trasformarlo in uno stadio del flusso.



**BAP-T06-01** transfer polarity = affirmative

### Lettura del grafico

Il percorso completo da source a destination, passando per il documento content, e' la projection di una sola proposition transfer. La linea tratteggiata verso il nodo giallo esprime soltanto la presenza del role opzionale behavior; non e' una BAProposition aggiuntiva e non introduce una seconda direzione di flusso.

## 8 Operatore produce

CURRENT / PRESERVED

### 8.1 Che cosa rappresenta

produce rappresenta l'asserzione governata che un actor o una capability **rende disponibile uno o piu' risultati/output**, eventualmente usando input esplicitamente governati. Il costrutto descrive il fatto produttivo minimo; non descrive automaticamente come il risultato viene calcolato, dove viene memorizzato o a chi viene successivamente trasferito.

#### Smallest semantic fact

Esiste un actor/capability A che rende disponibile almeno un risultato governato R. Eventuali input I appartengono alla stessa proposition solo quando la documentazione governa esplicitamente che vengano usati per quella produzione.

### 8.2 Quando usarlo

Usare produce quando la documentazione governa che un actor/capability produce, genera, calcola, restituisce, emette o rende disponibile un risultato identificabile. Il verbo concreto non e' decisivo: deve essere governato il significato "A rende disponibile R". Gli input sono opzionali e vengono inclusi soltanto quando la source li collega alla produzione del risultato.

### 8.3 Quando non usarlo

Non accettare una proposition produce quando il risultato deve essere inventato, quando e' nota soltanto l'esistenza di un dato senza un actor che lo renda disponibile, oppure quando un input viene aggiunto solo perche' compare altrove nell'architettura. La presenza di una sequenza di stage, di un componente o di una dipendenza temporale non dimostra da sola una produzione.

#### Gate minimo

Se la source non permette di identificare esattamente un actor e almeno un result, il risultato corretto e' INSUFFICIENT EVIDENCE / NOT SPECIFIED. L'assenza di input espliciti, invece, non invalida produce: input ha cardinalita' 0..\*.

## 8.4 Firma canonica

La firma completa e':

```
produce
  actor      -> BAREferent [1]
  input      -> BAREferent [0..*]
  result     -> BAREferent [1..*]
  polarity = affirmative | negative [1]
```

Gli scoped modifier `condition` e `temporalScope` compaiono soltanto quando la source limita realmente l'asserzione.

| Role   | Card. | Significato                                                                                                                 |
|--------|-------|-----------------------------------------------------------------------------------------------------------------------------|
| actor  | 1     | Actor/capability a cui la documentazione attribuisce il fatto di rendere disponibile il risultato.                          |
| input  | 0..*  | Significati che la documentazione dichiara usati dalla stessa produzione; non vengono ricostruiti da dipendenze plausibili. |
| result | 1..*  | Uno o piu' risultati/output governati resi disponibili dall'actor.                                                          |

## 8.5 Continuità di identità di input e result

Un `result` non riceve una nuova identità soltanto perché viene successivamente trasferito o usato come `input` da un altro comportamento. Quando il significato governato resta lo stesso, la BA riusa lo stesso `BAReferent`.

```
producer result    -> X
transfer content   -> X
consumer input     -> X
```

Allo stesso modo, un `input` di produce è il medesimo significato già identificato altrove quando non esiste una trasformazione semanticamente distinta.

## 8.6 Invarianti

- ogni produce accettato possiede esattamente un `actor` e almeno un `result`;
- `input` è opzionale: la proposition resta valida anche quando la `source` non governa `input` espliciti;
- `input` multipli o `result` multipli appartengono alla stessa proposition solo quando la stessa asserzione governata li lega allo stesso fatto produttivo;
- i `roleKey` sono operator-scoped: essere `result` in una proposition non assegna al referent un tipo universale “output”;
- `polarity` negativa richiede una negazione governata della produzione; assenza di `evidence` non equivale a produce negativo.

## 8.7 Forbidden inference

Dal solo produce non inferire algoritmo, modello, trasformazioni intermedie, `destination`, `transfer`, protocollo, serializzazione, `persistence`, `storage`, `ownership`, creazione ex novo dell’item, successo runtime, `retry`, `failure semantics` o completezza dello schema del risultato. Non aggiungere inoltre un `input` soltanto perché un dato è disponibile nel sistema o compare in uno stadio precedente.

## 8.8 Domande da porre alla documentazione

Le domande servono a restringere l'interpretazione; non sono un classificatore automatico.

1. La source governa realmente che qualcosa renda disponibile un risultato/output?
2. Qual e' esattamente l'actor? E' attribuito dalla documentazione senza inferenza architetturale?
3. Qual e' o quali sono i result prodotti?
4. La source governa uno o piu' input usati per quella produzione? Se non li governa, lasciarli assenti.
5. Actor, input e result hanno gia' un BAREferent con la stessa identita' semantica? Se si', riusarlo.
6. Piu' input o piu' result appartengono davvero allo stesso fatto produttivo oppure la documentazione governa produzioni distinte?
7. Un result mantiene la stessa identita' quando viene trasferito o riusato downstream? Se si', non creare un nuovo referent per il cambio di posizione.
8. La proposition vale sempre oppure solo sotto una condition o un temporal scope governato?
9. La source afferma la produzione o la nega esplicitamente? Non dedurre polarity negativa dall'assenza del risultato.
10. Rimane non stabilito l'actor o ogni result? Se si', fermarsi: il dato resta NOT SPECIFIED oppure diagnosis.

### Procedura minima

Leggere il smallest governed semantic fact; identificare o riusare l'actor e almeno un result; aggiungere soltanto gli input esplicitamente legati alla produzione; compilare una sola BAProposition minima; aggiungere scoped modifier solo se governati; infine controllare le forbidden inference e la continuita' di identita' downstream.

## 8.9 Esempio generico minimo

### Documentazione inventata ma coerente

“La capability RiskAssessmentComputation MUST usare il MeasurementRecord per rendere disponibili un RiskAssessment e una AssessmentConfidence.”

Assumendo che le identità abbiano superato il normale gate BA1, la proposition è’:

```
produce
  actor  -> RiskAssessmentComputation
  input  -> MeasurementRecord
  result -> RiskAssessment
  result -> AssessmentConfidence
  polarity -> affirmative
```

I due result restano nella stessa proposition perché la frase governa un unico fatto produttivo che li rende disponibili insieme.

## 8.10 Esempio corrente - DermaTriage Stage 3

Nella documentazione corrente, lo Stage 3 usa la ClinicalDescription prodotta dallo Stage 2 per il retrieval e rende disponibile il contesto dei casi storici recuperati. La proposition accettata corrente è’:

```
BAP-FR03-03-01
semanticOperatorKey = produce
  actor  -> HistoricalCaseRetrieval
  input  -> ClinicalDescription
  result -> RetrievedHistoricalCaseContext
polarity = affirmative
```

ClinicalDescription è’ lo stesso BAREferent già’ usato come content del transfer T06; non viene duplicato come “Stage3Input”. Il dettaglio ChromaDB / embedding appartiene ad altre proposition già’ separate e non viene incorporato nella shape di produce.

### Boundary example

“Il componente B dispone di X” non basta, da solo, a costruire produce: può’ descrivere disponibilità o stato senza governare chi produce X. Se actor o result non sono stabiliti, l’esito resta INSUFFICIENT EVIDENCE.

## 8.11 Prompt LLM di supporto all'autoring

Il prompt e' un ausilio operativo: non cambia la semantica BA e non rende autorevole una inferenza non supportata. Deve ricevere l'estratto documentale governato e, quando disponibile, il registro dei BAReferent gia' esistenti.

TASK: identify governed BA produce propositions.

Use ONLY the supplied documentation and existing BA registry.

A produce proposition exists only if the source supports:

actor [1], result [1..\*].

Input is optional: input [0..\*].

Rules:

1. Identify the smallest governed production fact: actor makes result available.
2. Reuse existing BAReferents when semantic identity is unchanged.
3. Propose a new BAReferent only when BA1 independent identity is needed.
4. Add input only when the source explicitly governs its use in this production.
5. Keep multiple inputs/results in one proposition only when the same governed production fact binds them to the same actor.
6. Preserve result identity when it later appears as transfer content or input.
7. Use polarity=negative only for an explicitly negated production assertion.
8. Add condition/temporalScope only when explicitly governed.
9. If actor or every result is missing, return INSUFFICIENT\_EVIDENCE.
10. Do not infer algorithm, implementation, destination, transfer, persistence, storage, creation, ownership, serialization or failure semantics.
11. Do not invent operators, roles, states or metadata.

Return EXACTLY the authoring wrapper and BA output shape shown next.

### Human review remains mandatory

L'LLM puo' proporre referent e proposition; l'accettazione richiede comunque verifica contro source, identity criterion, canonical naming e provenance. Una risposta sintatticamente completa non sana un gap documentale e non rende implicito un input non governato.

## 8.12 Output esatto atteso dall'estrazione assistita

Il wrapper seguente e' un formato di authoring/review, non un nuovo costrutto BA. La parte di output BA usa soltanto la shape ammessa.

```
PRODUCE_ASSESSMENT
  decision = ACCEPT_PRODUCE | NOT_A_PRODUCE | INSUFFICIENT_EVIDENCE
  evidence = <source locator / concise supported wording>
  notSpecified = [<only unresolved material items>]

BA_OUTPUT
  BAREferent reuse/candidates:
    <canonical names only as needed>

  BAProposition <candidate-id>
    semanticOperatorKey = produce
    actor -> <BAREferent>
    input -> <BAREferent>          # repeat for 0..*; omit if absent
    result -> <BAREferent>         # repeat for 1..*
    polarity = affirmative | negative
    scopedModifier = <condition|temporalScope> # omit if absent
```

Se `decision != ACCEPT_PRODUCE`, `BA_OUTPUT` non contiene una proposition produce accettabile. I nomi proposti restano candidate finche' non superano canonical naming, provenance e review. L'assenza di `input` non e' un errore quando la `source` governa `actor` e `result` ma non esplicita gli `input`.

## 8.13 Discriminazioni con altri costrutti

-

## 8.14 Rappresentazione Mermaid di produce

Per produce, la projection mette l'actor al centro del fatto produttivo. Gli eventuali input entrano nell'actor; i result escono dall'actor. L'intero insieme di frecce associato allo stesso proposition id rappresenta **una sola** BAPosition produce: le singole frecce visualizzano le participation della proposition e non creano proposition aggiuntive.

Nel worked example DermaTriage, ClinicalDescription e RetrievedHistoricalCaseContext sono mostrati con lo stesso simbolo documento bianco già usato per il contenuto informativo nella projection di transfer. Il role resta però quello della proposition corrente: rispettivamente input e result. La forma grafica non rinomina i role BA.

| Parte di produce          | Rappresentazione Mermaid                                                                                                                           |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| input                     | Nodo opzionale a sinistra; nel worked example informativo usa il simbolo documento bianco. Freccia direzionale verso l'actor. Ripetibile per 0..*. |
| actor                     | Nodo centrale che identifica l'actor/capability della production.                                                                                  |
| result                    | Nodo a destra; nel worked example informativo usa il simbolo documento bianco. Freccia in uscita dall'actor. Ripetibile per 1..*.                  |
| Proposition id / polarity | Metadati dell'intero pattern; non diventano nodi del flusso.                                                                                       |

```

flowchart LR
    I@{ shape: doc, label: "ClinicalDescription<br/>input" }
    A["HistoricalCaseRetrieval<br/>actor"]
    R@{ shape: doc, label: "RetrievedHistoricalCaseContext<br/>result" }

    I --> A --> R

    classDef actor fill:#EEF4F9,stroke:#275D86,color:#17324D
    classDef content fill:#FFFFFF,stroke:#275D86,color:#17324D
    class A actor
    class I,R content
  
```

Se input fosse assente, il nodo a sinistra e la relativa freccia verrebbero omissi. Con più input o result, i nodi vengono ripetuti e convergono/divergono dall'unico actor mantenendo lo stesso proposition id.

### 8.15 Esempio grafico corrispondente

Il diagramma seguente applica la convenzione alla proposition DermaTriage BAP-FR03-03-01. Il nodo centrale e' l'actor; l'input entra nell'actor e il result ne esce. I due referent informativi mantengono il simbolo documento bianco usato nella projection precedente. Il grafico non aggiunge destination, storage o dettagli di implementazione che produce non governa.



**BAP-FR03-03-01** produce polarity = affirmative

#### Lettura del grafico

L'intero pattern input -> actor -> result e' la projection di una sola proposition produce. Se la stessa proposition avesse piu' input o piu' result, comparirebbero piu' rami verso o dall'actor senza cambiare il numero di BAPosition rappresentate.

## 9 Operatore realize

CURRENT / PRESERVED

### 9.1 Che cosa rappresenta

`realize` rappresenta l'asserzione governata che uno o piu' significati concreti **realizzano o materializzano un significato piu' astratto** nella baseline considerata. Il costrutto preserva la distinzione tra *che cosa* il progetto richiede o governa a livello astratto e *quale elemento concreto* ne costituisce la realizzazione corrente.

#### Smallest semantic fact

Esiste un significato astratto A e almeno una realizzazione concreta R tale che la documentazione governa che R realizzi/materializzi A. I due significati restano distinti BAREferent: la relation non li fonde e non trasforma automaticamente l'abstract in una classe software.

### 9.2 Quando usarlo

Usare `realize` quando la documentazione distingue un significato astratto — per esempio capability, contract, behavior o strategia richiesta — da uno o piu' elementi concreti scelti per materializzarlo nella baseline. La relazione deve essere governata: la semplice presenza di una tecnologia dentro un componente non basta.

### 9.3 Quando non usarlo

Non usare `realize` per significare che un componente usa una libreria, legge dati da un database, dipende da un servizio, produce un output o trasferisce informazione. Questi fatti possono richiedere altre proposition e non diventano realization solo perche' il concreto compare nell'implementazione.

#### Gate minimo

Se la source identifica soltanto l'elemento concreto ma non il significato astratto che esso realizza, non inventare retroattivamente l'abstract per completare la BA. Registrare INSUFFICIENT EVIDENCE / NOT SPECIFIED e, quando materialmente utile, riportare il gap alla review documentale.

## 9.4 Firma canonica

La firma completa e':

```
realize
  abstract      -> BAREferent [1]
  realization   -> BAREferent [1..*]
  polarity      = affirmative | negative [1]
```

Gli scoped modifier `condition` e `temporalScope` compaiono soltanto quando la source limita realmente l'asserzione.

| Role        | Card. | Significato                                                                                           |
|-------------|-------|-------------------------------------------------------------------------------------------------------|
| abstract    | 1     | Significato astratto che la baseline governa indipendentemente dalla realizzazione concreta scelta.   |
| realization | 1..*  | Uno o piu' BAREferent concreti che la documentazione identifica come materializzazione dell'abstract. |

### Cardinalita' multipla non significa alternativa

La cardinalita' `realization [1..*]` permette a una proposition di riferirsi a piu' realization governate. Da questa cardinalita' non si deduce che esse siano alternative, intercambiabili, ridondanti o usate congiuntamente: tali distinzioni richiedono evidenza separata.

## 9.5 Continuità di identità tra abstract e realization

abstract e realization sono significati distinti e mantengono identità indipendente. La relation serve proprio a preservare contemporaneamente il significato stabile e la scelta concreta della baseline.

```
abstract capability -> ClinicalDescriptionGeneration  
realization concrete -> Qwen2-VL-7B-Instruct
```

La realization non sostituisce il BAREferent astratto. Se altri documenti qualificano, vincolano o riusano ClinicalDescriptionGeneration, tali proposition continuano a riferirsi all'abstract e non vengono automaticamente trasferite alla tecnologia concreta.

## 9.6 Invarianti

- ogni realize accettato possiede esattamente un abstract e almeno una realization;
- abstract e realization restano BAREferent distinti e riusabili;
- più realization appartengono alla stessa proposition soltanto quando la stessa asserzione governata le lega allo stesso abstract;
- la relation deve esprimere materializzazione del significato astratto, non mera presenza tecnica o collaborazione;
- polarity negativa richiede una negazione governata della realization; assenza di evidence non equivale a realize negativo.

## 9.7 Forbidden inference

Dal solo realize non inferire subclassing, ereditarietà software, equivalenza semantica completa, sostituibilità drop-in, compatibilità di interfaccia, deployment topology, ownership, dipendenza, consumo di servizio, flusso dati, produzione, transfer, algoritmo interno, modello di sicurezza o completezza delle realization possibili. La projection può ricordare una gerarchia abstract/concrete, ma non importa automaticamente la semantica UML delle classi.

## 9.8 Domande da porre alla documentazione

Le domande cercano la distinzione abstract/concrete prima della sintassi BA.

1. Qual e' il significato astratto che la documentazione governa indipendentemente dalla tecnologia o strategia corrente?
2. Quale elemento concreto la source dichiara che realizza, implementa o materializza quel significato?
3. Se rimuovo il nome della tecnologia concreta, l'abstract resta un significato comprensibile e governato? Se no, verificare che non stiamo inventando un livello astratto.
4. Il concreto realizza realmente l'abstract oppure lo usa, lo invoca, ne dipende, gli fornisce input o ne riceve output?
5. Abstract e realization hanno gia' BAReferent con la stessa identita' semantica? Se si', riusarli.
6. Se compaiono piu' realization, la documentazione le lega davvero allo stesso abstract nella stessa asserzione? Non inferire alternative o composizione.
7. La source governa condition o temporal scope della realization?
8. La source afferma o nega esplicitamente la realization? Non dedurre polarity negativa dall'assenza di una tecnologia.
9. La tecnologia e' nominata ma la funzione astratta resta implicita? Fermarsi e registrare un possibile gap documentale invece di completarlo per plausibilita'.

### Procedura minima

Isolare prima il significato astratto; verificare poi che la source identifichi esplicitamente almeno una realizzazione concreta; riusare le identita' BA esistenti; compilare una sola proposition minima; infine riesaminare che nessuna relazione di uso, dipendenza, produzione o trasferimento sia stata compressa impropriamente dentro `realize`.

## 9.9 Esempio generico minimo

### Documentazione inventata ma coerente

“La capability `DocumentClassification` MUST essere realizzata nella baseline corrente dalla strategia `TransformerClassificationStrategy`.”

Assumendo che le identità abbiano superato il normale gate BA1, la proposition è:

```
realize
  abstract      -> DocumentClassification
  realization   -> TransformerClassificationStrategy
  polarity      -> affirmative
```

Il fatto preservato è la scelta concreta rispetto a un significato astratto già distinguibile; non vengono inferiti modello specifico, formato degli input o contratto degli output.

## 9.10 Esempio corrente - DermaTriage Stage 2

La documentazione corrente distingue la capability `ClinicalDescriptionGeneration` dalla realizzazione concreta usata nello Stage 2. L'analisi corrente rappresenta quindi:

```
BAP-FR03-02-02
semanticOperatorKey = realize
  abstract      -> ClinicalDescriptionGeneration
  realization   -> Qwen2-VL-7B-Instruct
polarity = affirmative
```

Questa proposition non sostituisce la proposition produce che governa la produzione di `ClinicalDescription`. `realize` risponde a “quale elemento concreto materializza la capability?”, mentre produce preserva il fatto che la capability rende disponibile il risultato.

### Boundary example

“Il componente A usa `TechnologyB`” non basta a costruire `realize`. Senza un abstract governato e una relazione di materializzazione esplicita, l'uso della tecnologia resta un fatto diverso oppure un gap da riesaminare nella documentazione.

## 9.11 Prompt LLM di supporto all'autoring

Il prompt non deve creare un abstract soltanto per rendere sostituibile o piu' elegante una tecnologia gia' nominata. Un gap documentale resta un gap.

TASK: identify governed BA realize propositions.

Use ONLY the supplied documentation and existing BA registry.

A realize proposition exists only if the source supports:

abstract [1], realization [1..\*].

Rules:

1. Identify the smallest governed abstract/concrete materialization fact.
2. Reuse existing BAREferents when semantic identity is unchanged.
3. Propose a new BAREferent only when BA1 independent identity is needed.
4. Accept realization only when the source governs that the concrete referent realizes/materializes the abstract referent.
5. Do not convert mere use, dependency, invocation, production or data flow into realization.
6. Keep multiple realizations in one proposition only when the same governed assertion binds them to the same abstract.
7. Do not infer alternatives, replaceability, composition or completeness.
8. Use polarity=negative only for an explicitly negated realization assertion.
9. Add condition/temporalScope only when explicitly governed.
10. If abstract or every realization is missing, return INSUFFICIENT\_EVIDENCE.
11. If a concrete technology is named but its abstract function is not governed, report the documentation gap; do not invent the abstraction.
12. Do not invent operators, roles, states or metadata.

Return EXACTLY the authoring wrapper and BA output shape shown next.

### Human review remains mandatory

La distinzione fra "realizza" e "usa" e' semanticamente materiale. L'LLM puo' proporla, ma l'accettazione richiede verifica contro la source e contro le altre proposition gia' estratte, soprattutto quando la tecnologia partecipa anche a produzione, servizio o transfer.

## 9.12 Output esatto atteso dall'estrazione assistita

Il wrapper seguente e' un formato di authoring/review, non un nuovo costruito BA. La parte di output BA usa soltanto la shape ammessa.

```

REALIZE_ASSESSMENT
  decision = ACCEPT_REALIZE | NOT_A_REALIZE | INSUFFICIENT_EVIDENCE
  evidence = <source locator / concise supported wording>
  documentationGap = <abstract function missing | none>
  notSpecified = [<only unresolved material items>]

BA_OUTPUT
  BAREferent reuse/candidates:
    <canonical names only as needed>

  BAProposition <candidate-id>
    semanticOperatorKey = realize
    abstract -> <BAREferent>
    realization -> <BAREferent> # repeat for 1..*
    polarity = affirmative | negative
    scopedModifier = <condition|temporalScope> # omit if absent

```

Se `decision != ACCEPT_REALIZE`, `BA_OUTPUT` non contiene una proposition `realize` accettabile. `documentationGap` appartiene al wrapper di review e non alla BA: serve soltanto a rendere visibile quando una tecnologia concreta e' documentata senza che sia governata la capability astratta che ci si aspetterebbe di poter sostituire o riesaminare.

## 9.13 Discriminazioni con altri costrutti

-

## 9.14 Rappresentazione Mermaid di realize

La projection usa una disposizione verticale simile alla relazione abstract/concrete: l'abstract e' sopra, la realization concreta sotto e una linea tratteggiata con triangolo vuoto punta verso l'abstract. La convenzione rende immediata la lettura "il concreto realizza l'astratto" senza trasformare la BA in un modello di classi.

| Parte di realize          | Rappresentazione Mermaid                                                         |
|---------------------------|----------------------------------------------------------------------------------|
| abstract                  | Nodo superiore, marcato graficamente come significato astratto.                  |
| realization               | Nodo inferiore; ripetibile per 1..*.                                             |
| Relazione                 | Linea tratteggiata con triangolo vuoto rivolto verso l'abstract e label realize. |
| Proposition id / polarity | Metadati dell'intero pattern; non diventano nodi della gerarchia.                |

```
classDiagram
direction BT

class ClinicalDescriptionGeneration {
  <<abstract>>
}

class Qwen2VL7BInstruct {
  <<realization>>
}

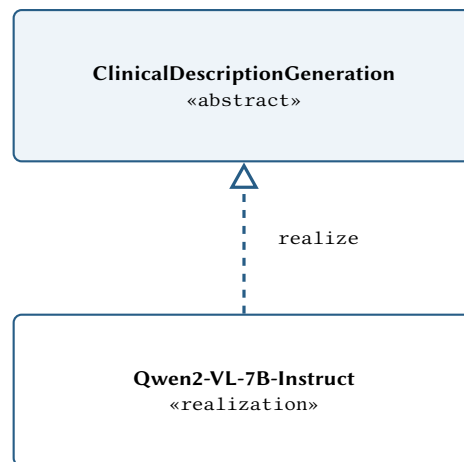
Qwen2VL7BInstruct ..|> ClinicalDescriptionGeneration : realize
```

### Somiglianza grafica non equivale a subclassing

Il triangolo e la disposizione verticale sono una convenzione di projection per rendere visibile la relation abstract/concrete. Non autorizzano inferenze di ereditarieta', type hierarchy, interface conformance o object-oriented design.

### 9.15 Esempio grafico corrispondente

Il diagramma applica la convenzione alla proposition `DermaTriage BAP-FR03-02-02`. `ClinicalDescriptionGeneration` resta il significato astratto; `Qwen2-VL-7B-Instruct` e' la realizzazione concreta della baseline. Il grafico non rappresenta input, output o transfer: questi appartengono ad altre proposition.



**BAP-FR03-02-02** realize polarity = affirmative

#### Lettura del grafico

La freccia si legge dal concreto verso l'astratto: `Qwen2-VL-7B-Instruct` realizza `ClinicalDescriptionGeneration`. Se una stessa proposition contenesse piu' realization governate, piu' rami tratteggiati convergerebbero sullo stesso abstract mantenendo un unico proposition id; il grafico non dichiarerebbe da solo se tali realization siano alternative o congiunte.

## 10 Operatore constrain

CURRENT / PRESERVED

### 10.1 Che cosa rappresenta

`constrain` rappresenta una **restrizione governata, riusabile e interrogabile** applicata a un significato identificato. Il costrutto serve quando il progetto non si limita a nominare un dato, un comportamento o uno stato, ma ne governa anche un contratto, un dominio ammesso, una proprietà richiesta o un'altra limitazione stabile che deve poter essere recuperata e analizzata separatamente.

#### Smallest semantic fact

Esiste un target governato T e almeno una restriction C tale che la documentazione stabilisce che T sia soggetto a C. La restriction può essere un significato governato con identità propria oppure una struttura locale controllata; in entrambi i casi il target resta esplicito.

### 10.2 Quando usarlo

Usare `constrain` quando la restriction deve restare distinguibile e interrogabile: per esempio un data/content contract, un allowed domain, una proprietà obbligatoria, un requisito di confidenzialità applicato a un delivery o un medium imposto a uno specifico comportamento di trasferimento.

### 10.3 Quando non usarlo

Non usare `constrain` per descrivere quale valore possiede una singola occorrenza runtime, quale ramo di una regola decisionale viene scelto, quale elemento viene selezionato, chi produce o trasferisce un dato, oppure quale meccanismo esegue materialmente il controllo. La restriction qualifica il target: non sostituisce il comportamento che produce, trasporta, seleziona o verifica.

#### Gate minimo

Prima di introdurre `constrain`, identificare il target esatto. Se non è chiaro se la restriction appartenga al contenuto trasferito, al behavior di transfer, a un contratto riusabile o a un altro significato governato, non propagare il vincolo per convenienza: mantenere il gap esplicito e tornare alla documentazione quando necessario.

## 10.4 Firma canonica

La firma top-level resta unica:

```

constrain
  constraintTarget -> BAREferent [1]
  constraintValue  -> controlled value/structure [1..*]
  polarity         = affirmative | negative [1]

```

Gli scoped modifier `condition` e `temporalScope` compaiono soltanto quando la source limita realmente la restriction.

| Role                          | Card. | Significato                                                                                                 |
|-------------------------------|-------|-------------------------------------------------------------------------------------------------------------|
| <code>constraintTarget</code> | 1     | BAREferent il cui significato e' ristretto dalla proposition.                                               |
| <code>constraintValue</code>  | 1..*  | Restriction governata: puo' essere un BAREferent con identita' propria oppure una forma locale controllata. |

## 10.5 Due modi di esprimere la restriction

**A. Constraint nominato / riusabile.** Quando il contratto o la restriction possiede identita' analitica indipendente:

```
constraintValue -> <BAREferent>
```

Esempio: `ClinicalDescriptionContentContract`. Il nome non e' un'etichetta grafica: e' un BAREferent riusabile e change-addressable.

**B. Constraint locale.** Quando non serve una nuova identita' BA, la restriction resta inline. Puo' essere un dominio locale controllato, oppure una struttura di proprieta':

```

constraintValue
  property  -> <controlled semantic key> [1]
  presence  -> required [0..1]
  vocabulary -> [<controlled typed local value | BAREferent> 1..*] [0..1]

```

Un allowed domain semplice puo' essere espresso direttamente come `constraintValue` locale, senza creare un BAREferent artificiale soltanto per ospitare l'elenco.

## 10.6 Continuità' di identità e livello del constraint

Il `constraintTarget` mantiene la propria identità indipendentemente dalla restriction. Se `constraintValue` è un `BAReferent` nominato, anche il constraint mantiene identità separata e può a sua volta essere target di altre proposition.

```
ClinicalDescription
  constrained by -> ClinicalDescriptionContentContract

ClinicalDescriptionContentContract
  constrained by -> structured local restrictions
```

Questa catena permette di distinguere *a quale dato si applica il contratto* da *come il contratto è composto*. Una modifica della struttura interna del contratto non richiede di rinominare il dato o di duplicare tutte le proposition che vi fanno riferimento.

## 10.7 Invarianti

- ogni proposition accettata possiede esattamente un `constraintTarget` e almeno un `constraintValue`;
- un `BAReferent` usato come constraint nominato richiede normale giustificazione BA1 di identità indipendente;
- `property` e i valori locali non diventano automaticamente `BAReferent`;
- `presence -> required` afferma la presenza della proprietà; la sola `vocabulary` non la implica;
- più `constraintValue` restano nella stessa proposition soltanto se la stessa asserzione governata li applica allo stesso target e scope;
- una restriction sul content non si propaga automaticamente al behavior di transfer, e una restriction sul behavior non modifica automaticamente lo schema del content.

## 10.8 Forbidden inference

Dal solo `constraint` non inferire il valore runtime osservato, validazione riuscita, enforcement mechanism, algoritmo, ordine di valutazione, selection, mapping decisionale, produzione, transfer, ownership, responsibility, encryption effettivamente applicata o conformità di una specifica occorrenza. Un `allowed domain` non dice quale valore verrà scelto; un `data contract` non dice da solo chi invia o riceve il dato.

## 10.9 Domande da porre alla documentazione

Le domande servono prima a localizzare la restriction e soltanto dopo a scegliere la sua forma.

1. Qual e' il significato esatto che viene ristretto? Il target e' un dato/content, un behavior di transfer, uno stato, un contratto o altro BAREferent?
2. La source governa realmente una restriction stabile e interrogabile oppure descrive soltanto un dettaglio locale senza utilita' analitica autonoma?
3. La restriction possiede un nome/identita' riusabile nel progetto? Se si', esiste o serve un BAREferent dedicato?
4. Se la restriction resta locale, e' un dominio diretto oppure una proprieta' strutturata?
5. Per una proprieta' strutturata, qual e' il property key governato?
6. La source richiede che la proprieta' sia presente? Usare presence -> required solo in questo caso.
7. La source governa un insieme chiuso di valori ammessi? Inserirlo in vocabulary senza dedurre automaticamente la presenza.
8. Il vincolo riguarda il dato stesso oppure il modo in cui quel dato viene trasferito? Non spostare il target tra content e transfer behavior.
9. Piu' restriction appartengono davvero alla stessa asserzione e allo stesso scope oppure devono restare proposition separate?
10. La restriction vale sempre oppure soltanto sotto condition/temporal scope governati?
11. La source afferma o nega esplicitamente il constraint? Non dedurre polarity negativa dall'assenza di un valore o di una proprieta'.
12. Se property, domain, scope o target non sono governati, lasciare NOT SPECIFIED / diagnosis invece di completarli per plausibilita'.

### Procedura minima

Identificare il target; determinare se la restriction richiede identita' propria oppure resta locale; compilare soltanto i dettagli effettivamente governati; verificare la separazione content/transfer behavior; infine riesaminare che la proposition non stia simulando una regola decisionale, una selection o un runtime fact.

## 10.10 Esempio generico minimo

### Documentazione inventata ma coerente

“Ogni TelemetryRecord trasferito tra i componenti MUST rispettare il TelemetryRecordContract. Il contratto richiede rappresentazione JSON e la presenza dei campi eventId e timestamp.”

Assumendo identità BA già giustificate:

```
constrain
  constraintTarget -> TelemetryRecord
  constraintValue  -> TelemetryRecordContract
  polarity = affirmative

constrain
  constraintTarget -> TelemetryRecordContract
  constraintValue
    property -> representation
    presence -> required
    vocabulary -> [JSON]
  constraintValue
    property -> eventId
    presence -> required
  constraintValue
    property -> timestamp
    presence -> required
  polarity = affirmative
```

## 10.11 Esempio corrente - DermaTriage Stage 2

La documentazione corrente stabilisce che ClinicalDescription abbia un contratto di contenuto riusabile e che tale contratto sia testuale, composto da cinque parti richieste.

```
BAP-FR03-02-03 constrain
  constraintTarget -> ClinicalDescription
  constraintValue  -> ClinicalDescriptionContentContract

BAP-FR03-02-04 constrain
  constraintTarget -> ClinicalDescriptionContentContract
  constraintValue -> representation required in [TEXTUAL]
  constraintValue -> partCount required in [5]
  constraintValue -> shape required
  constraintValue -> borders required
  constraintValue -> color required
  constraintValue -> texture required
  constraintValue -> suspiciousFeatures required
```

Il primo fatto identifica *quale contratto governa il dato*; il secondo rende interrogabile *come quel contratto è composto*. I due livelli non vengono fusi.

## 10.12 Prompt LLM di supporto all'autoring

Il prompt deve preservare sia il target corretto sia il livello del constraint. In particolare non deve spostare una restriction dal dato al transfer behavior o viceversa soltanto per rendere il diagramma piu' comodo.

TASK: identify governed BA constrain propositions.

Use ONLY the supplied documentation and existing BA registry.

A constrain proposition exists only if the source supports:  
constraintTarget [1], constraintValue [1..\*].

Rules:

1. Identify the smallest governed reusable/queryable restriction fact.
2. Identify the exact target before extracting the restriction.
3. Reuse existing BAreferents when semantic identity is unchanged.
4. Use a named constraint BAreferent only when independent BA1 identity is justified; otherwise keep the restriction local.
5. For local structured constraints, use only governed property, presence and vocabulary information. Do not invent schema fields.
6. presence=required only when the source explicitly requires presence.
7. vocabulary does not imply presence.
8. Preserve the distinction between content constraints and transfer-behavior constraints. Do not move a restriction between them without source support.
9. Do not turn runtime values, validation results, decision mappings, selection, production or transfer semantics into constrain.
10. Keep multiple constraintValue entries together only when they share the same governed target and assertion scope.
11. Use polarity=negative only for an explicitly negated restriction assertion.
12. Add condition/temporalScope only when explicitly governed.
13. If target or every restriction value is missing, return INSUFFICIENT\_EVIDENCE.
14. Do not invent operators, roles, states, BAreferents or metadata.

Return EXACTLY the authoring wrapper and BA output shape shown next.

### Human review remains mandatory

L'LLM puo' riconoscere la forma del constraint, ma la review umana deve verificare soprattutto identity promotion, scope e placement: lo stesso vocabolo tecnico puo' qualificare il content, il behavior di transfer o un contratto nominato con effetti analitici diversi.

### 10.13 Output esatto atteso dall'estrazione assistita

Il wrapper distingue la decisione di authoring dalla BA. constraintForm serve alla review e non introduce un nuovo field BA.

```

CONSTRAIN_ASSESSMENT
  decision = ACCEPT_CONSTRAIN | NOT_A_CONSTRAINT | INSUFFICIENT_EVIDENCE
  evidence = <source locator / concise supported wording>
  constraintForm = NAMED_REFERENT | LOCAL_DIRECT | LOCAL_STRUCTURED
  targetKind = CONTENT | TRANSFER_BEHAVIOR | OTHER
  documentationGap = <material unresolved meaning | none>
  notSpecified = [<only unresolved material items>]

BA_OUTPUT
  BAREferent reuse/candidates:
    <canonical names only as needed>

  BAProposition <candidate-id>
    semanticOperatorKey = constrain
    constraintTarget -> <BAREferent>
    constraintValue -> <BAREferent | controlled local value/domain>
      # OR repeat local structured values as governed:
      property -> <controlled semantic key>
      presence -> required # omit if absent
      vocabulary -> [<controlled values>] # omit if absent
    polarity = affirmative | negative
    scopedModifier = <condition|temporalScope> # omit if absent

```

Se decision != ACCEPT\_CONSTRAIN, BA\_OUTPUT non contiene una proposition constrain accettabile. constraintForm e targetKind sono soltanto controlli di authoring per evitare che la scelta della rappresentazione alteri il significato governato.

### 10.14 Discriminazioni con altri costrutti

-

## 10.15 Rappresentazione Mermaid di constrain

La projection mantiene visibile il `constraintTarget` e colloca la restriction **vicino al significato che qualifica**. Un `constraint` nominato usa un nodo dedicato; una restriction locale strutturata puo' essere resa come nota/card collegata al target. La linea tratteggiata etichettata `constrain` visualizza una `BAProposition` distinta e non modifica le frecce di data flow.

| Target del constraint         | Placement nella view                                                                                                                    |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Content / dato trasferito     | Constraint accanto al nodo documento/content. Source e destination continuano a leggere lo stesso <code>BASReferent</code> informativo. |
| Transfer behavior             | Constraint accanto al <code>BASReferent</code> del behavior/segmento di transfer, non al content.                                       |
| Constraint nominato           | Nodo riusabile collegato al target con linea tratteggiata <code>constrain</code> .                                                      |
| Constraint locale strutturato | Nota/card vicina al target, con le sole property/presence/vocabulary governate.                                                         |

```

flowchart LR
    S["ClinicalDescriptionGeneration<br/>source"]
    C@{ shape: doc, label: "ClinicalDescription<br/>content" }
    D["HistoricalCaseRetrieval<br/>destination"]
    K["ClinicalDescriptionContentContract<br/>named constraint"]
    F["TEXTUAL; 5 parts; required fields<br/>structured constraint"]

    S --> C --> D
    C -. ->|constrain| K
    K -. ->|constrain| F

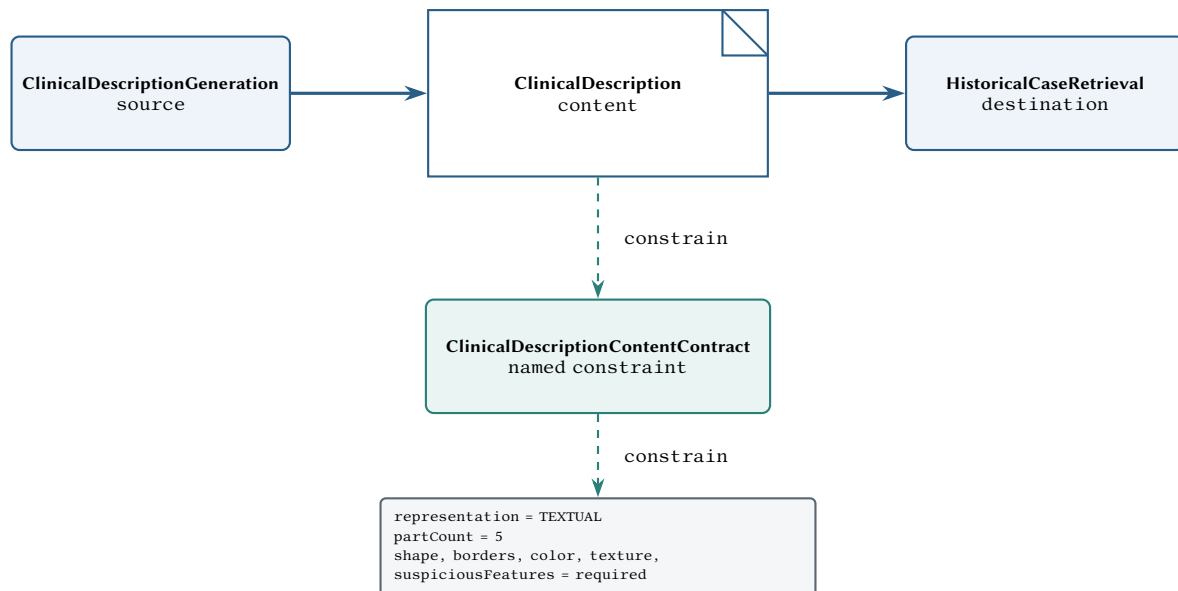
    classDef endpoint fill:#EEF4F9,stroke:#275D86,color:#17324D
    classDef content fill:#FFFFFF,stroke:#275D86,color:#17324D
    classDef contract fill:#EAF5F3,stroke:#2A7F78,color:#17324D
    classDef detail fill:#F4F6F8,stroke:#5B6770,color:#17324D
    class S,D endpoint
    class C content
    class K contract
    class F detail
  
```

### Placement non equivale a propagazione

Mostrare un data contract vicino a un transfer non significa che il transfer stesso sia il `constraintTarget`. Il target resta il content quando la restriction descrive forma o dominio del dato; diventa il transfer behavior soltanto quando la source governa una restriction sul delivery, per esempio medium o confidentiality del segmento.

## 10.16 Esempio grafico corrispondente

Il diagramma combina senza fonderle tre proposition: il transfer di `ClinicalDescription`, il constraint che associa il dato al `ClinicalDescriptionContentContract` e il constraint che descrive il contratto. Il data path resta `source -> content -> destination`; le linee tratteggiate appartengono a proposition constrain separate.



### Lettura del grafico

Chi produce e chi riceve condividono lo stesso `ClinicalDescription`; il contratto rende visibile che cosa quel dato deve rispettare. Se una restriction riguardasse invece il delivery — per esempio il medium o la confidentiality del trasferimento — il nodo di constraint verrebbe collegato al `BAREferent` del transfer behavior e non a `ClinicalDescription`.

## 11 Scoped modifier condition

CURRENT / PRESERVED

### 11.1 Che cosa rappresenta

`condition` è uno **scoped modifier BA2** che limita localmente l'applicabilità di una `BAProposition`. Il fatto espresso dall'operatore e dai suoi `participant` non cambia identità: la `condition` stabilisce soltanto *quando quella specifica asserzione si applica*. Non è un nuovo top-level operator e non è una decision rule abbreviata.

#### Smallest semantic fact

Esiste una `BAProposition` governata `P` e una condizione locale `C` tale che `P` è asserita soltanto quando `C` vale nello scope documentato. Eliminando `condition` si perderebbe il confine di applicabilità della proposition, non il significato del suo operatore.

### 11.2 Quando usarlo

Usare `condition` quando la source governa un guard/applicability scope per una singola proposition e quel significato non richiede identità, provenance, participant o riuso autonomi. La condizione deve essere source-grounded; non può essere dedotta soltanto perché un comportamento sembra plausibile in uno scenario.

### 11.3 Quando non usarlo

Non usare `condition` per codificare un mapping IF/THEN/ELSE che determina quale risultato viene prodotto, per una dependency/prerequisite, per un constraint riusabile o per nascondere un significato che deve diventare `BAResultant`/`BAProposition` autonomo.

#### Gate minimo

La domanda non è "c'è un if nella frase?", ma "la documentazione limita quando questa stessa proposition vale?". Se la `condition` seleziona risultati, introduce participant propri o deve essere riusata indipendentemente, fermarsi e riesaminare il costruito.

## 11.4 Forma canonica nella BAProposition

La grammatica BA2 preservata ammette `scopedModifier` `[0..*]` e, nel lower bound corrente, soltanto i `kind condition` e `temporalScope`. R7 rende esplicita `condition` con la seguente forma di authoring:

```
BAProposition
  semanticOperatorKey = <operator>
  participation ...
  polarity = affirmative | negative

  scopedModifier
    condition -> <source-grounded local applicability statement>
```

La parte destra di `condition` e' contenuto locale controllato della proposition, non un `BAResult` implicito. La serializzazione deve preservare la formulazione canonica scelta durante la review senza inventare property, boolean o stati non governati.

| Elemento                    | Card.          | Significato                                                                                                                         |
|-----------------------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>scopedModifier</code> | 0..*           | Modifier locali ammessi dalla BAProposition; il lower bound corrente contiene <code>condition</code> e <code>temporalScope</code> . |
| <code>condition</code>      | per instance 1 | Guard/applicability statement governato per la proposition che lo contiene.                                                         |

### Nessun DSL logico generale

La forma locale non introduce automaticamente AND, OR, truth table, property model o evaluation order. Se la source governa una costruzione decisionale che seleziona un risultato, il candidato da riesaminare e' `decisionRule`, non una `condition` sempre piu' complessa.

## 11.5 Locality / identity boundary

Una condition resta embedded soltanto finché modifica localmente una proposition e non richiede identità analitica indipendente. BA2 R2/R3 preserva il promotion test: se il significato viene riusato, separatamente revisionato, diventa participant-bearing o deve essere target di altre asserzioni, si applicano i normali criteri BA1 invece di nascondere nel modifier.

LOCAL

```
produce(...)
condition -> LesionImage unavailable
```

PROMOTION PRESSURE

```
NoImageCondition must be reused / constrained / traced independently
-> evaluate BAREferent / BAProposition identity under BA1
```

## 11.6 Invarianti

- la condition qualifica l'intera BAProposition, non un singolo edge scelto dal renderer;
- l'operatore, i roleKey e i BAREferent della proposition mantengono la stessa identità;
- una condition locale non crea automaticamente un nuovo BAREferent;
- l'assenza della condition opposta non autorizza a inventare un ramo ELSE;
- l'ordine testuale di più facts o view non implica evaluation order;
- la condition deve essere governata dalla source allo stesso livello documentale che giustifica la proposition o da evidence tracciabile compatibile con tale scope.

## 11.7 Forbidden inference

Da condition non inferire detection logic, boolean property, stato runtime persistente, branch complementare, fallback, precedence, sequencing, retry/failure semantics, dependency, constraint riusabile o mapping di risultato. "Image unavailable" non autorizza da solo a inventare `LesionImage.available = FALSE`.

## 11.8 Domande da porre alla documentazione

1. Quale BAProposition e' limitata dalla condizione? Il modifier deve avere un owner propositionale preciso.
2. La source sta dicendo *quando la proposition vale* oppure *come una condizione determina quale risultato viene scelto*?
3. La condition e' esplicita/governata oppure la stiamo deducendo da un percorso ritenuto probabile?
4. Il significato puo' restare locale senza BAREferent autonomo?
5. E' riusato, separatamente revisionato, participant-bearing o target di altre proposition? Se si', applicare l'identity test BA1.
6. La formulazione richiederebbe di inventare property, boolean, enum o stato? Se si', non completarli per plausibilita'.
7. Esiste davvero un ramo opposto governato? Una condition non implica automaticamente ELSE o il complemento logico.
8. La source governa una dependency/prerequisite? In tal caso verificare dependOn, non condition.
9. La source governa una restriction riusabile? In tal caso verificare constrain.
10. La condition e' temporale? Se descrive before/after/during/until, verificare temporalScope invece di condition.

### Procedura minima

Estrarre prima la BAProposition senza modifier; verificare che il fatto sia semanticamente completo; isolare poi il guard governato; applicare locality/identity test; aggiungere condition soltanto se cambia l'applicabilita' della stessa proposition. Infine controllare che non stia simulando decisionRule, dependOn o constrain.

## 11.9 Esempio generico minimo

### Documentazione inventata ma coerente

“Quando la modalita’ di manutenzione e’ attiva, SystemMonitor produce un MaintenanceAlert a partire da SensorStatus.”

Assumendo i BAReferent gia’ giustificati:

```
produce
  actor   -> SystemMonitor
  input   -> SensorStatus
  result  -> MaintenanceAlert
  polarity = affirmative
  scopedModifier
    condition -> MaintenanceMode active
```

La condition dice quando questa produzione si applica. Non afferma che MaintenanceMode sia un boolean, non crea un ramo per la modalita’ inattiva e non descrive come viene rilevato lo stato.

## 11.10 Esempio corrente - DermaTriage symptom-only

La documentazione R25 stabilisce: “Quando l’immagine della lesione non e’ disponibile, DermaTriage determina l’urgenza del caso utilizzando le informazioni sintomatologiche disponibili associate allo stesso caso.”

```
produce
  actor   -> DermaTriage
  input   -> SymptomInformation
  result  -> CaseUrgency
  polarity = affirmative
  scopedModifier
    condition -> LesionImage unavailable
```

Non serve materializzare NoImageAvailableCondition come BAReferent finche’ il significato resta locale. Il percorso image-based con immagine disponibile deve essere supportato da proposition/documentazione proprie; non viene inventato come ELSE di questa proposition.

## 11.11 Prompt LLM di supporto all'autoring

TASK: identify governed proposition-local condition modifiers.

Use ONLY the supplied documentation, accepted BA and current BA authority.

A condition modifier exists only when the source limits when an otherwise complete BAProposition applies.

Rules:

1. Identify the complete BAProposition first: operator, roles and referents.
2. Extract condition only if the source explicitly governs the applicability guard.
3. The condition qualifies the whole proposition, not an arbitrary visual edge.
4. Keep the condition local when it has no independent identity/reuse.
5. Do not create a BAREferent only to name a one-use guard.
6. If the condition is reused, separately reviewed, participant-bearing or targeted by other propositions, stop and request BA1 identity review.
7. Do not invent boolean properties, enums, states or opposite conditions.
8. Do not infer ELSE, fallback or complementary behavior from one condition.
9. Do not use condition for IF/THEN/ELSE result construction; review decisionRule.
10. Do not use condition for prerequisite/dependency; review dependOn.
11. Do not use condition for reusable restrictions; review constrain.
12. Do not infer temporal order; temporal applicability belongs to temporalScope.
13. If the guard is not source-grounded, return INSUFFICIENT\_EVIDENCE.
14. Do not invent operators, roles, BAREferents or metadata.

Return EXACTLY the authoring wrapper and BA output shape shown next.

### Human review remains mandatory

La review umana deve verificare soprattutto il confine fra applicabilit  locale e semantica autonoma. La forma linguistica "quando/se" e' soltanto un indizio: non decide automaticamente condition, decisionRule o dependOn.

## 11.12 Output esatto atteso dall'estrazione assistita

```

CONDITION_ASSESSMENT
  decision = ACCEPT_CONDITION | NOT_A_CONDITION | INSUFFICIENT_EVIDENCE
  evidence = <source locator / concise supported wording>
  ownerProposition = <candidate-id / identified governed fact>
  identityPressure = NONE | REVIEW_BA1
  notSpecified = [<only unresolved material items>]

BA_OUTPUT
  BAREferent reuse/candidates:
    <canonical names only as needed>

  BAProposition <candidate-id>
    semanticOperatorKey = <existing operator>
    <operator-scoped roles and terms>
    polarity = affirmative | negative
    scopedModifier
      condition -> <source-grounded local applicability statement>

```

ownerProposition e identityPressure appartengono al wrapper di review, non alla BA.

## 11.13 Discriminazioni con i costrutti vicini

| Confine                      | Discriminatore                                                                                                          |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| condition /<br>decisionRule  | condition cambia quando la proposition si applica; decisionRule governa come condizioni/input determinano un risultato. |
| condition /<br>constrain     | condition e' locale all'asserzione; constrain e' una restriction riusabile/interrogabile sul proprio target.            |
| condition /<br>dependOn      | condition e' guard di applicabilita'; dependOn afferma prerequisite/dependency e supporta impact propagation.           |
| condition /<br>BAREferent    | il modifier resta locale; se il significato richiede identita'/riuso autonomi, applicare BA1.                           |
| condition /<br>temporalScope | condition governa un guard non temporale; before/after/during/until appartengono al temporal scope.                     |

## 11.14 Rappresentazione Mermaid di condition

La *condition* qualifica l'intera *BAProposition*. La *projection* canonica la rende come un **tag di condition** colorato, posto sopra il ramo governato. Il tag non e' un *BAREferent*, non e' un input dell'operatore e non e' un nodo decisionale: visualizza soltanto lo *scopedModifier.condition* della *proposition owner* o del blocco di vista selezionato per quel ramo.

| Semantica BA                                | Regola di projection                                                                                                                                                                                                                       |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>condition modifier</i><br>Applicabilita' | Un tag ellittico colorato con label canonica <condition>.<br>Il tag viene posto in testa al ramo governato; la sua posizione rende visibile l'applicabilita' locale senza introdurre dataflow o nuovi role BA.                             |
| Owner block                                 | Nelle viste composte il ramo puo' essere rappresentato con <i>un solo blocco owner</i> per lato, quando l'obiettivo e' mostrare la separazione di <i>condition</i> e non l'espansione completa di tutte le <i>proposition</i> sottostanti. |
| Due condition distinte                      | Producono due tag distinti; nessun ramo viene inventato come complemento dell'altro.                                                                                                                                                       |
| Decision rule                               | Il rombo decisionale resta riservato alla <i>projection</i> di <i>decisionRule</i> ; <i>condition</i> non usa il rombo e non introduce automaticamente una forma IF/ELSE.                                                                  |

```

flowchart LR
    C1([Condition A])
    B1["Owner block A"]

    C2([Condition B])
    B2["Owner block B"]

    C1 --> B1
    C2 --> B2

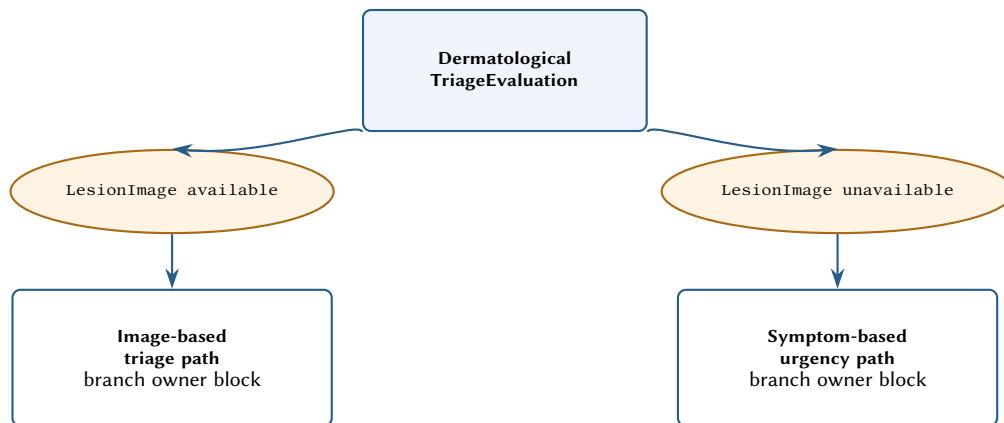
```

### Semantic determinism della projection

Con la stessa BA accettata e lo stesso *projection* profile devono restare invariati: numero dei tag di *condition*, shape ellittica, label canonica della *condition* e ramo *owner*. La posizione geometrica dei nodi puo' variare: in questa fase non richiediamo layout determinism. Se la vista usa un solo blocco *owner* per ramo, anche quella scelta deve essere esplicita e coerente con lo scope della *view*.

### 11.15 Esempio grafico corrispondente

Il diagramma mostra sullo stesso foglio due rami con condition distinte. I tag colorati LesionImage available e LesionImage unavailable stanno sopra i rispettivi rami. Per mantenere la vista leggibile, ogni lato della condition e' rappresentato con **un solo blocco owner** invece di espandere tutto cio' che sarebbe ulteriormente derivabile dalla documentazione.



condition = LesionImage available      condition = LesionImage unavailable

#### Lettura del grafico

Il tag colorato indica soltanto la condition del ramo. Non e' un nodo decisionale e non implica automaticamente un ramo complementare. In questa vista ciascun lato e' compresso in un solo blocco owner: la projection evidenzia il fatto che due condition diverse governano due rami diversi, ma non espande automaticamente l'intera struttura interna dei rami.

## 12 Operator structure decisionRule

CANDIDATE / NOT ADMITTED

### 12.1 Che cosa rappresenta

decisionRule e' qui una **candidate operator structure** locale a una BAProposition produce. Rappresenta il caso in cui la source non governa soltanto che un actor produce un result da uno o piu' input, ma governa anche *come condizioni o valori degli input selezionano il valore del result*. La candidate non rinomina il costrutto: riesamina soltanto la collocazione dello storico decisionRule. Actor, input e result restano quelli dell'owner produce; la struttura decisionale viene annidata.

#### Smallest semantic fact

Esiste una proposition produce gia' completa e una regola governata R che determina quale valore del suo result viene selezionato a partire dagli input/condizioni documentati. Eliminando decisionRule si conserva il fatto che il result viene prodotto, ma si perde il mapping operativo governato che seleziona il valore del result.

### 12.2 Quando usarla

Usare la candidate quando la documentazione esplicita un mapping del tipo "in queste condizioni, il result assume questo valore" e actor/input/result coincidono semanticamente con quelli di una production gia' identificata. La regola deve essere governata: non basta che un mapping sia plausibile o implementato nel codice.

### 12.3 Quando non usarla

Non usarla per dire soltanto quando la proposition si applica: quello resta `scopedModifier.condition`. Non usarla per dichiarare soltanto il dominio ammesso del result: quello resta `constrain`. Non usarla per dependency, sequencing o fallback non governati.

#### Delta rispetto all'autorità corrente

R3 mantiene decisionRule come top-level operator con propri role actor/input/result. R7 non cambia tale authority: testa se lo stesso decisionRule possa essere rilocato come operatorStructure dell'owner produce, evitando la duplicazione di actor/input/result. La candidate richiede regression e promotion esplicita.

## 12.4 Forma canonica candidate dentro produce

La forma di rebuild mantiene la proposition produce come owner del fatto produttivo e aggiunge al massimo una struttura locale di selezione del risultato:

```
BAProposition
  semanticOperatorKey = produce
  participation
    actor  -> <BAREferent> [1]
    input  -> <BAREferent> [0..*]
    result -> <BAREferent> [1..*]
  polarity = affirmative | negative
  scopedModifier [0..*]
    condition -> <local applicability guard>
  operatorStructure [0..1]
  decisionRule
    branch [1..*]
      when -> <source-grounded selection condition> [1]
    resultAssignment [1..*]
      target -> <result BAREferent>
      value  -> <controlled local value | BAREferent>
```

| Elemento          | Card.        | Significato                                                                                                                |
|-------------------|--------------|----------------------------------------------------------------------------------------------------------------------------|
| operatorStructure | 0..1         | Struttura locale dell'owner proposition; non crea da sola una seconda BAProposition.                                       |
| decisionRule      | per owner 1  | Mapping governato che seleziona il valore del result.                                                                      |
| branch            | 1..*         | Branch esplicitamente governati dalla source.                                                                              |
| when              | per branch 1 | Condizione di selezione source-grounded; puo' preservare una residual condition esplicita senza inventare un ELSE globale. |
| resultAssignment  | 1..*         | Assegnazione governata a un result gia' partecipante all'owner produce.                                                    |

### Vocabolario delle comparison ancora sotto pressione

L'autorità corrente di decisionRule.comparison ammette equals | notEquals. DermaTriage governa esplicitamente confidence > 0.85 e quindi mette sotto pressione l'aggiunta candidate di greaterThan. Non generalizzare automaticamente a >=, <, range o aritmetica senza nuovi casi governati.

## 12.5 Locality / identity boundary

Nella candidate, il decisionRule annidato condivide l'identità della BAProposition produce che qualifica. Non viene creato un secondo fact con gli stessi actor/input/result soltanto per ospitare il mapping. Questo è precisamente il delta da verificare contro l'attuale decisionRule top-level dell'authority.

```
OWNER FACT
  produce(actor, input(s), result)

LOCAL STRUCTURE
  operatorStructure.decisionRule
    -> how the governed result value is selected
```

Se la regola deve avere provenance, review/change lifecycle, riuso o targetability realmente indipendenti dall'owner produce, la rilocalizzazione locale non va forzata: il caso riapre l'identità/BA2 design e può falsificare questa candidate.

## 12.6 Invarianti

- actor, input e result sono quelli della proposition produce; la struttura non li duplica;
- condition e decisionRule possono coesistere: la prima governa quando la production si applica, la seconda quale valore del result viene selezionato una volta applicabile;
- ogni branch deve essere source-grounded; un branch mancante non viene inventato come ELSE;
- una residual condition come “gli altri casi HIGH” resta limitata allo scope governato e non diventa un default globale;
- l'ordine dei branch non implica precedence a meno che la source non governi un ordine necessario;
- il mapping non modifica il dominio ammesso del result se tale dominio è governato separatamente da constrain.

## 12.7 Forbidden inference

Non inferire completezza del mapping, default, precedence, short-circuit, gestione degli input mancanti, coercion dei tipi, range non documentati, error/failure handling o nuovi comparison key. Non trasformare valori locali come P1 in BAREferent se non superano l'identity test BA1.

## 12.8 Domande da porre alla documentazione

1. Esiste già un fatto produce completo con actor, input e result governati?
2. La source sta dicendo *quando la production vale* o *quale valore del result viene selezionato*?
3. Le condizioni del mapping sono esplicite e governate, oppure servirebbe inventare property o soglie?
4. I branch producono valori dello stesso result della proposition owner?
5. Il dominio dei valori è governato separatamente e richiede anche un constrain?
6. Esistono branch residuali espliciti (“altri casi”, “otherwise”)? Qual è il loro scope esatto?
7. I branch sono semanticamente disgiunti oppure servirebbe precedence/evaluation order non documentato?
8. Le comparison richieste appartengono al vocabolario corrente? Se no, registrare pressure invece di inventare operatori di confronto.
9. La regola deve essere riusata, revisionata o tracciata indipendentemente dall’owner produce? Se sì, fermare la rilocalizzazione locale e riaprire identity/BA2 review.
10. La source governa davvero un mapping di risultato oppure soltanto una dependency, restriction o classification?

### Procedura minima

Estrarre prima produce senza rule; stabilizzare actor/input/result; separare eventuale condition di applicabilità; isolare poi soltanto i branch che selezionano il result; preservare i valori e le condizioni governate; verificare branch residuali e comparison vocabulary; infine confrontare la collocazione candidate annidata con l’attuale decisionRule top-level per assicurare che nessuna identità o provenance necessaria venga persa.

## 12.9 Esempio generico minimo

### Documentazione inventata ma coerente

“QueueClassifier produce QueueClass da RequestClassification. Se classKind e' EXPRESS, il risultato e' FAST; se classKind e' STANDARD, il risultato e' NORMAL.”

```
produce
  actor  -> QueueClassifier
  input  -> RequestClassification
  result -> QueueClass
  polarity = affirmative
  operatorStructure
    decisionRule
      branch
        when -> RequestClassification.classKind equals EXPRESS
        resultAssignment -> QueueClass = FAST
      branch
        when -> RequestClassification.classKind equals STANDARD
        resultAssignment -> QueueClass = NORMAL
```

L'assenza di altri branch non autorizza a inventare un default.

## 12.10 Pressure case corrente - DermaTriage

FR-MR01-04-01 governa quattro selezioni: HIGH + confidence > 0.85 -> P1; gli altri casi HIGH -> P2; MEDIUM -> P3; LOW -> P4. La seguente e' una *candidate authoring view*, non BA gia' accettata:

```
produce
  actor  -> OperationalPriorityDerivation    [candidate]
  input  -> CaseUrgency
  input  -> TriageConfidence
  result -> OperationalTriagePriority
  scopedModifier
    condition -> LesionImage available
  operatorStructure
    decisionRule
      HIGH + confidence > 0.85 -> P1
      HIGH, other applicable cases -> P2
      MEDIUM -> P3
      LOW -> P4
```

Il caso forza due review separate: greaterThan nel comparison vocabulary e una forma multi-branch che preservi “altri casi HIGH” senza trasformarla in ELSE globale.

## 12.11 Prompt LLM di supporto all'autoring

TASK: identify governed result-selection logic for an existing produce proposition. Use ONLY supplied documentation, accepted BA and current BA authority.

This R7 nesting of decisionRule is CANDIDATE and does not replace current top-level  
 ↪ decisionRule authority.

Rules:

1. Identify the complete produce proposition first: actor, input(s), result(s),  
 ↪ polarity.
2. Separate proposition applicability from result selection.
3. Put applicability in scopedModifier.condition, never in operatorStructure.  
 ↪ decisionRule.
4. Identify nested decisionRule only if the source governs how conditions/input values select a value of an existing produce result.
5. Reuse the produce actor/input/result. Do not duplicate them as a second fact.
6. Extract only source-governed branches and assignments.
7. Do not invent ELSE/default/completeness/precedence/evaluation order.
8. Preserve explicit residual wording such as "other HIGH cases" in its exact scope.  
 ↪
9. Use only admitted comparison forms; if the source requires a new comparison (for example greaterThan), return VOCABULARY\_PRESSURE rather than inventing it.
10. Do not turn allowed-domain semantics into result selection; review constrain.
11. If the rule needs independent identity/reuse/provenance/change lifecycle, return IDENTITY\_PRESSURE and stop local nesting.
12. Do not invent BAreferents, properties, typed values or metadata.

Return EXACTLY the review wrapper and candidate BA shape shown next.

### Human review remains mandatory

La review umana deve verificare due confini distinti: condition contro result selection e local structure contro independent proposition identity. La presenza linguistica di "if" non decide nessuno dei due confini.

## 12.12 Output esatto atteso dall'estrazione assistita

### RESULT\_SELECTION\_ASSESSMENT

```

decision = ACCEPT_CANDIDATE | NOT_RESULT_SELECTION | INSUFFICIENT_EVIDENCE
evidence = <source locator / concise supported wording>
ownerProduce = <candidate-id / identified governed fact>
authorityStatus = CANDIDATE_RELOCATION_OF_DECISION_RULE
comparisonPressure = NONE | VOCABULARY_PRESSURE
identityPressure = NONE | REVIEW_BA1_BA2
notSpecified = [<only unresolved material items>]

```

### BA\_OUTPUT\_CANDIDATE

```

BAProposition <owner-id>
  semanticOperatorKey = produce
  <produce roles and terms>
  polarity = affirmative | negative
  scopedModifier [only if independently source-grounded]
    condition -> <applicability guard>
  operatorStructure
    decisionRule
      <source-grounded branches and result assignments>

```

I campi di assessment appartengono al wrapper di review, non alla BA.

## 12.13 Discriminazioni con i costrutti vicini

| Confine                                      | Discriminatore                                                                                                                                                                              |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| decisionRule / condition                     | la rule seleziona il valore del result; condition cambia quando l'intera proposition si applica.                                                                                            |
| decisionRule / constrain                     | la rule governa la selezione runtime/operativa; constrain governa un dominio o restriction senza dire come un valore viene scelto.                                                          |
| decisionRule / produce                       | produce resta il fatto owner; la rule aggiunge dettaglio di selezione senza creare un secondo actor/input/result.                                                                           |
| nested decisionRule / top-level decisionRule | e' lo stesso costrutto canonico: l'authority corrente lo rappresenta come operator top-level; la candidate R7 ne testa soltanto la rilocalazione come operatorStructure dell'owner produce. |
| decisionRule / dependOn                      | la rule seleziona un risultato; dependOn dichiara prerequisite/dependency e non simula branch IF/THEN.                                                                                      |

## 12.14 Rappresentazione Mermaid della operator structure

Quando una view sceglie di renderizzare `decisionRule`, la struttura appare come **annotazione grigia a bordo tratteggiato** associata alla proposition produce. Non e' un `BAReferent`, non e' un secondo actor e non crea un nuovo dataflow edge.

| Semantica BA | Regola di projection candidate                                                                                                                            |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| produce      | Mantiene il normale pattern blu <code>input(s) -&gt; actor -&gt; result(s)</code> .                                                                       |
| condition    | Se inclusa dalla view, resta il tag ambra sopra il ramo/proposition owner.                                                                                |
| decisionRule | Box grigio chiaro, bordo tratteggiato, collegamento di qualificazione senza arrowhead di dataflow.                                                        |
| Visibility   | Il projection profile puo' omettere completamente il box quando la decision logic non interessa alla view. L'omissione non crea uno stato BA "collapsed". |
| Dettaglio    | Se renderizzata, la view puo' mostrare i branch governati necessari al proprio scope; non puo' cambiarne significato o inventarne altri.                  |

flowchart LR

```

I1["Input A"] --> A["Actor"] --> R["Result"]
I2["Input B"] --> A
C([Condition]) -. applies .- A
RS["decisionRule\nbranch 1 ... \nbranch 2 ..."] -.- A

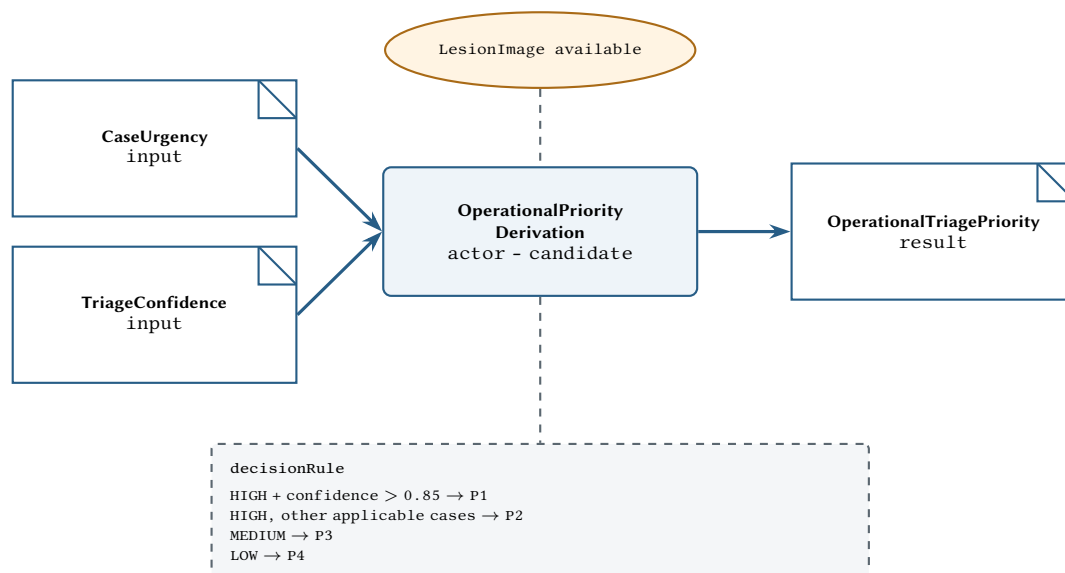
```

### Semantic determinism e omissione controllata

A parita' di BA e projection profile, la decisione di includere o omettere `operatorStructure` deve essere deterministica. Se inclusa, branch/label/assignment derivano dalla BA; se omessa, la BA resta invariata e la view dichiara l'omissione tramite il proprio coverage contract. Nessun modello "collapsed" viene creato.

## 12.15 Esempio grafico corrispondente

La figura usa soltanto il produce necessario al pressure test DermaTriage. Non espande Stage 1–4 o altre proposition. Il box grigio tratteggiato e' struttura interna renderizzata; una view che non necessita della decision logic puo' ometterlo completamente.



### Lettura della candidate view

Blu = produzione; ambra = applicability; grigio tratteggiato = result selection interna. Il collegamento grigio non e' dataflow. La figura non promuove la rilocalizzazione: visualizza il decisionRule annidato candidate da sottoporre a regression prima di modificare l'autorità BA.

## A Materiale temporaneo di rebuild

### APPENDICE TEMPORANEA DI REBUILD - DA ELIMINARE PRIMA DELLA GUIDA DEFINITIVA

Il contenuto di questa appendice governa *come stiamo ricostruendo la guida*, non il funzionamento della Base Analysis. Deve restare disponibile durante il rebuild per garantire coerenza, genealogia e regression, ma va rimosso dalla guida definitiva dopo il consolidamento. Se una sua regola deve sopravvivere come metodologia BA, deve essere trasferita esplicitamente nella parte metodologica destinata alla guida definitiva prima della rimozione.

### A.1 Scopo, lineage e confine della ricostruzione

Questa guida non nasce per dichiarare che la revisione più recente sia automaticamente la migliore. Nasce per rendere leggibile e applicabile il patrimonio BA già costruito, riesaminandolo in modo uniforme mentre viene applicato alla documentazione DermaTriage corrente.

#### Regola di costruzione cumulativa

La nuova struttura editoriale può essere diversa da R3, Core R42 o Complete R42. Il significato precedente, però, non può scomparire per effetto della riscrittura. Ogni distinzione precedente deve essere preservata, raffinata, rilocalizzata, ritestata, respinta con motivazione oppure lasciata esplicitamente aperta.

### A.2 Tre livelli da non confondere

| Livello            | Esempio                                                                   | Effetto                                                                            |
|--------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Authority corrente | BA0–BA5 + Operational Guide R3                                            | Governa il metodo corrente finché non esiste promotion esplicita.                  |
| Research evidence  | R42, operator review, ledger, pressu-<br>re, candidate, vecchi<br>appunti | Può motivare chiarimenti o cambi, ma non modifica da solo il metodo.               |
| Rebuild candidate  | questa guida                                                              | Superficie cumulativa di review; diventa authority solo con checkpoint successivo. |

#### Nessun target di cardinalità

Il fatto che la BA corrente abbia quattordici operatori top-level non crea un obiettivo di quattordici operatori. Anche il precedente snapshot a tredici elementi resta genealogia utile, non un target da ripristinare. Il numero finale deve essere un risultato della necessità semantica e della regression, non una quota.

## APPENDICE TEMPORANEA DI REBUILD - DA ELIMINARE PRIMA DELLA GUIDA DEFINITIVA

Il contenuto di questa appendice governa *come stiamo ricostruendo la guida*, non il funzionamento della Base Analysis. Deve restare disponibile durante il rebuild per garantire coerenza, genealogia e regression, ma va rimosso dalla guida definitiva dopo il consolidamento. Se una sua regola deve sopravvivere come metodologia BA, deve essere trasferita esplicitamente nella parte metodologica destinata alla guida definitiva prima della rimozione.

### A.3 Corpus di ricostruzione e stati epistemici

Il rebuild usa come input controllato il registro DDTA\_R25\_BASE\_ANALYSIS\_REBUILD\_SOURCE\_REGISTER\_R1.md. Prima di chiudere un costrutto vanno ricercati non solo il suo nome corrente, ma anche alias storici, pressure collegate, candidate construct, alternative rifiutate e counterexample.

#### Recency non equivale ad authority

R42 Core/Complete e' una superficie di lettura aggiornata e molto utile per il rebuild. R3 e BA0-BA5 restano tuttavia authority corrente. La guida nuova puo' usare R42 come base esplicativa senza promuovere automaticamente le sue candidate o chiudere le sue open pressure.

### A.4 Etichette epistemiche minime

| Etichetta                | Uso                                                       | Disciplina                                                                        |
|--------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------|
| CURRENT / PRESERVED      | Semantica corrente preservata                             | Deve restare compatibile con l'authority o dichiararne esplicitamente il delta.   |
| CANDIDATE / NOT ADMITTED | Costrutto/evoluzione con evidence positiva ma non ammessa | Non usare come vocabulary corrente senza admission esplicita.                     |
| OPEN                     | Problema reale non chiuso                                 | Mantenere il gap visibile; non completare per convenienza.                        |
| REJECTED                 | Ipotesi testata e respinta nello scope disponibile        | Conservare ragione, test e condizioni di eventuale reopen.                        |
| REBUILD CONTRACT         | Regola editoriale del rebuild                             | Governa come descriviamo e confrontiamo i costrutti durante questa ricostruzione. |

#### Regola anti-perdita

Una guida piu' corta non e' una guida migliore se per ottenere la sintesi elimina una distinzione ancora utile. Il rebuild puo' comprimere la prosa soltanto dopo aver dimostrato che la distinzione resta ricostruibile e tracciabile.

## APPENDICE TEMPORANEA DI REBUILD - DA ELIMINARE PRIMA DELLA GUIDA DEFINITIVA

Il contenuto di questa appendice governa *come stiamo ricostruendo la guida*, non il funzionamento della Base Analysis. Deve restare disponibile durante il rebuild per garantire coerenza, genealogia e regression, ma va rimosso dalla guida definitiva dopo il consolidamento. Se una sua regola deve sopravvivere come metodologia BA, deve essere trasferita esplicitamente nella parte metodologica destinata alla guida definitiva prima della rimozione.

## A.5 Construct Description Contract

### REBUILD CONTRACT

Prima di riscrivere il primo costrutto BA, fissiamo il modo in cui *tutti* i costrutti verranno descritti. Questo contratto serve a impedire che la guida cambi struttura in funzione del caso del momento e rende confrontabili operatori current, condition form, strutture candidate, pressure e ipotesi rifiutate.

#### Invariante editoriale

Ogni costrutto usa la stessa sequenza descrittiva. Un campo puo' essere NOT APPLICABLE, NOT FROZEN o OPEN; non puo' essere omesso silenziosamente. Se un costrutto futuro dimostra che il contratto non basta, si modifica prima il contratto e poi si riallineano i costrutti gia' trattati.

#### Controllo rigoroso, esposizione naturale

Il CDC e' un contratto di *copertura*, non un template rigido di impaginazione. I campi richiesti devono essere tutti trattati e ricostruibili durante la review, ma la guida definitiva puo' combinarli in paragrafi e sezioni piu' naturali quando questo non nasconde informazioni, stato epistemico o confini semantici. Struttura di controllo rigorosa non significa esposizione meccanica.

## APPENDICE TEMPORANEA DI REBUILD - DA ELIMINARE PRIMA DELLA GUIDA DEFINITIVA

Il contenuto di questa appendice governa *come stiamo ricostruendo la guida*, non il funzionamento della Base Analysis. Deve restare disponibile durante il rebuild per garantire coerenza, genealogia e regression, ma va rimosso dalla guida definitiva dopo il consolidamento. Se una sua regola deve sopravvivere come metodologia BA, deve essere trasferita esplicitamente nella parte metodologica destinata alla guida definitiva prima della rimozione.

### A.6 Direzione obbligatoria della spiegazione

```
human semantic idea
  -> BA term / construct class
  -> formal shape
  -> boundaries and discriminators
  -> examples
  -> evidence and genealogy
  -> open questions
  -> current rebuild disposition
```

#### Da evitare

- iniziare dalla firma e lasciare al lettore il compito di indovinarne il significato;
- usare l'esempio DermaTriage come definizione implicita del costrutto;
- aggiungere campi solo a un costrutto perche' il suo caso concreto ne ha bisogno;
- completare la firma di una candidate non ammessa per rendere il template piu' ordinato;
- nascondere rejected hypothesis o pressure per ottenere una sezione apparentemente "chiusa".

### A.7 Revisione del contratto

Il contratto parte come CDC-R1. Ogni sua modifica futura deve avere un counterexample, una motivazione generalizzabile e una regression sui costrutti gia' descritti.

## APPENDICE TEMPORANEA DI REBUILD - DA ELIMINARE PRIMA DELLA GUIDA DEFINITIVA

Il contenuto di questa appendice governa *come stiamo ricostruendo la guida*, non il funzionamento della Base Analysis. Deve restare disponibile durante il rebuild per garantire coerenza, genealogia e regression, ma va rimosso dalla guida definitiva dopo il consolidamento. Se una sua regola deve sopravvivere come metodologia BA, deve essere trasferita esplicitamente nella parte metodologica destinata alla guida definitiva prima della rimozione.

### A.8 Construct Description Contract - template, parte 1

| N. | Campo                       | Contenuto richiesto                                                                                                                           |
|----|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Canonical name              | Nome usato dal rebuild. Alias storici vanno nel campo genealogy, non come nomi paralleli current.                                             |
| 2  | Construct class             | Top-level operator, condition form, local reusable structure, candidate, open pressure, rejected hypothesis o altro meccanismo BA dichiarato. |
| 3  | Epistemic status            | Stato esplicito: current/preserved, candidate/not admitted, open, rejected oppure altro stato definito.                                       |
| 4  | Owning BA layer / contract  | Dove il significato appartiene: BA0, BA1, BA2, BA3, BA4, BA5 oppure confine cross-layer esplicito.                                            |
| 5  | Human definition            | Spiegazione in linguaggio naturale di che cosa rappresenta il costrutto.                                                                      |
| 6  | Smallest semantic fact      | Il nucleo minimo che si perde eliminando il costrutto.                                                                                        |
| 7  | When to use                 | Condizioni positive che giustificano l'uso. Devono riferirsi a significato governato, non a parole chiave.                                    |
| 8  | When not to use             | Casi in cui il costrutto sembra plausibile ma imporrebbe semantica sbagliata o eccessiva.                                                     |
| 9  | Canonical syntax / shape    | Forma corrente se ammessa. Per candidate non chiusa: NOT FROZEN salvo working shape esplicitamente marcata.                                   |
| 10 | Roles, types, cardinalities | Ruoli e vincoli formali ammessi. Nessun role viene inventato per l'esempio corrente.                                                          |

#### Semantica prima della sintassi

I campi 5–8 devono poter essere compresi anche da chi non conosce ancora la sintassi BA. I campi 9–10 formalizzano il significato; non lo sostituiscono.

## APPENDICE TEMPORANEA DI REBUILD - DA ELIMINARE PRIMA DELLA GUIDA DEFINITIVA

Il contenuto di questa appendice governa *come stiamo ricostruendo la guida*, non il funzionamento della Base Analysis. Deve restare disponibile durante il rebuild per garantire coerenza, genealogia e regression, ma va rimosso dalla guida definitiva dopo il consolidamento. Se una sua regola deve sopravvivere come metodologia BA, deve essere trasferita esplicitamente nella parte metodologica destinata alla guida definitiva prima della rimozione.

### A.9 Construct Description Contract - template, parte 2

| N. | Campo                               | Contenuto richiesto                                                                                   |
|----|-------------------------------------|-------------------------------------------------------------------------------------------------------|
| 11 | Semantic invariants                 | Cio' che deve restare vero in ogni uso valido del costruito.                                          |
| 12 | Forbidden inferences                | Fatti che la presenza del costruito non autorizza a dedurre.                                          |
| 13 | Closest constructs / discriminators | Confini pairwise con i costrutti piu' facili da confondere.                                           |
| 14 | Minimal generic positive example    | Esempio didattico minimo, separato dall'evidence di progetto.                                         |
| 15 | DermaTriage application example     | Uso reale sulla documentazione corrente, quando disponibile e supportato.                             |
| 16 | Negative / boundary example         | Caso in cui il costruito va rifiutato o usato con limite preciso.                                     |
| 17 | Genealogy and research evidence     | Authority, candidate guide, operator review, PR/CC/CL/CMD/OBS e precedenti rilevanti.                 |
| 18 | Open pressures / candidate deltas   | Questioni ancora vive che non devono essere nascoste dalla descrizione principale.                    |
| 19 | Review / acceptance questions       | Domande ripetibili che permettono a un autore di verificare l'uso.                                    |
| 20 | Current rebuild disposition         | Cosa decide questa revisione: preserve, refine, retest, reject, hold, candidate admission later, ecc. |

#### Change protocol del CDC

Se un costruito non entra nel template, non si crea una eccezione locale. Si registra il counterexample, si verifica se il problema e' generale, si produce una revisione CDC-R2 e si riesaminano retroattivamente i costrutti gia' descritti. Nessuna modifica implicita del formato e' ammessa.

#### Candidate con firma non chiusa

Per provideService il template resta completo, ma i campi "Canonical syntax" e "Roles, types, cardinalities" possono dichiarare NOT FROZEN. E' preferibile una lacuna esplicita a una firma inventata solo per uniformare l'impaginazione.

## APPENDICE TEMPORANEA DI REBUILD - DA ELIMINARE PRIMA DELLA GUIDA DEFINITIVA

Il contenuto di questa appendice governa *come stiamo ricostruendo la guida*, non il funzionamento della Base Analysis. Deve restare disponibile durante il rebuild per garantire coerenza, genealogia e regression, ma va rimosso dalla guida definitiva dopo il consolidamento. Se una sua regola deve sopravvivere come metodologia BA, deve essere trasferita esplicitamente nella parte metodologica destinata alla guida definitiva prima della rimozione.

### A.10 Roadmap temporanea del rebuild

1. usare CDC-R1 come controllo di copertura, senza trasformarlo in una scheda editoriale rigida;
2. assumere le Sezioni 1-12 come base espositiva corrente della Rebuild R7;
3. preservare governed semantic fact, BAREferent, BAProposition, identity criterion, stopping rule e structural-feedback boundary come fondazioni già riesaminate;
4. considerare transfer, produce, realize e constrain i primi quattro operatori completamente ricostruiti, condition il primo scoped modifier completamente ricostruito e decisionRule la prima operatorStructure candidate riesaminata sotto lo stesso CDC-R1;
5. mantenere come placeholder le discriminazioni pairwise dei quattro operatori dove questa scelta resta deliberata; per condition e la candidate annidata decisionRule, conservare invece i discriminatori necessari fra applicability, result selection, allowed-domain constraint, dependency e identity pressure;
6. preservare in constrain la distinzione fra constraint nominato/riusabile e restriction locale, e nella projection distinguere il vincolo sul content dal vincolo sul transfer behavior;
7. continuare la Base Analysis di MR-01 dalla documentazione DermaTriage corrente consolidata in R25; usare FR-MR01-04-01 come pressure case per la rilocalizzazione di decisionRule senza trattare la candidate come authority prima della regression;
8. per il prossimo costrutto effettivamente necessario, partire dalle guide precedenti, ricostruire prima il metodo generale e poi verificarlo con evidence corrente DermaTriage;
9. mantenere prompt LLM e wrapper di authoring separati dalla semantica BA: possono evolvere senza creare nuovi campi o operatori;
10. solo dopo BA accettata per uno scope, produrre e verificare le projection downstream previste.

#### Stop point di questa R7

La Rebuild R7 contiene la ricostruzione completa dei quattro operatori transfer, produce, realize, constrain e dello scoped modifier condition. La Sezione 12 aggiunge una candidate strutturale: preservare il mapping di risultato come operatorStructure.decisionRule dell'owner produce, invece di duplicare actor/input/result in una seconda top-level proposition decisionRule. Il delta non è ammesso: deve ancora superare la regression sul caso Facial Access, il pressure test DermaTriage e un checkpoint esplicito.

## Indice di integrita' delle pagine

Ogni pagina possiede un MD5 del proprio contenuto canonico; il valore stampato e' escluso dal payload e l'hash della pagina indice usa <SELF> per evitare dipendenza circolare.

| Pag. | Contenuto                                                                    | MD5 contenuto                     |
|------|------------------------------------------------------------------------------|-----------------------------------|
| 1    | Frontespizio della Base Analysis Guide Rebuild R7                            | 0c456e68bcd056124768219bacdcab68  |
| 2    | 1 - Che cos' e' la Base Analysis                                             | fa01b7db301f77cd9988907daf2b3397  |
| 3    | 1 - Perché esiste e principi di base                                         | 3b620cf21e74f64e397656ec9f3038da  |
| 4    | 1 - Direzione dell' authority e feedback                                     | b12d8f94dc04c833c0d916453255f9be  |
| 5    | 1 - Minimum sufficient BA, NOT SPECIFIED e gap                               | 44c236d6809ab29df535fe29ff00352d  |
| 6    | 2 - BA View / Projection Contract e source types                             | b9514c3f03b3db13c46c35ba6ae647130 |
| 7    | 2 - Mermaid e campi minimi della view                                        | db02dca9f3877577832e39efb2e6b7c1  |
| 8    | 2 - Coverage mode e famiglie di viste                                        | b4b3bc661001ccb16d9acdef681806bd  |
| 9    | 2 - View gap e mapping discipline                                            | dcd3d21eb4eb28971b488dcd1c7cf01   |
| 10   | 3 - Dal testo documentale al significato governato                           | 9fdff9a0139ea36ea577008c81fb109f  |
| 11   | 3 - Governed semantic fact                                                   | f0c4184f4f1ebcb7ce1ab9394c5c5209  |
| 12   | 3 - Segmentazione, ambiguita e livello documentale                           | e3200807fcbf2aa290f0748b9c099b69  |
| 13   | 4 - Elementi fondamentali della Base Analysis                                | b9028ef062953f4b7ab73bf7ac7975c   |
| 14   | 4 - BAReferent: identita analitica riusabile                                 | 849a9ae9b63b6f5b84acbf3c1015a920  |
| 15   | 4 - BAReferent: esempi DermaTriage e confini                                 | c6fca40abd8cf6a72634f4b1d07fa450  |
| 16   | 4 - BAProposition: asserzione analitica identificabile                       | 83b4a5ba93a8eb218e7a0816bb7e023a  |
| 17   | 4 - BAReferent e BAProposition insieme                                       | 8063825cd44fd393ef91b9a5957e86    |
| 18   | 5 - Granularita, minimalita e costruzione della BA                           | f23c4cd3c7776e49cd0794a9ecfef9a9  |
| 19   | 6 - Indipendenza tra struttura documentale e struttura BA                    | bc3e4197b5ba6fc299035dd3ee55f0fa  |
| 20   | 6 - Structural feedback e no authority inversion                             | 90bad45a1dd4546a84ffcaafe19514f   |
| 21   | 6 - DermaTriage DEC02 DEC04 come pressure case                               | a4a5a63a56b233dc9685ef8ade259ac3  |
| 22   | 6 - Downstream Utility Test e riallineamento BA                              | 0b13ca345d9009ea1359e515bbdbf161  |
| 23   | 7 - transfer: definizione, smallest fact e criteri d'uso                     | 833af8b0be91d76075d6f8d588152778  |
| 24   | 7 - transfer: firma canonica, ruoli e cardinalita                            | e3df0dae1ea4b8fc206e94586671531d  |
| 25   | 7 - transfer: identity continuity, invarianti e forbidden inference          | 7f9e91357a2287afcb0bcd8cdc5d05cad |
| 26   | 7 - transfer: domande di identificazione e procedura di estrazione           | ca23e6876cbd5a9485a716d5b3e2a2a7  |
| 27   | 7 - transfer: esempio generico e worked example DermaTriage corrente         | 5b77098bebe4ae19463db3e451b2de    |
| 28   | 7 - transfer: prompt LLM per l'identificazione controllata                   | f59b8e5e2e26e94598d971d526ab5349  |
| 29   | 7 - transfer: output BA atteso e discriminazioni differite                   | daa47270d6924161b6041ddc1f28526b  |
| 30   | 7 - transfer: rappresentazione Mermaid                                       | fb9f9f38b0f615ee2066c351a264262d  |
| 31   | 7 - transfer: esempio grafico Mermaid                                        | 124230f8d8f7900c16fc39a88a25fa5c  |
| 32   | 8 - produce: definizione, smallest fact e criteri d'uso                      | a04b76a31302078a1ca5e5411d643f1e  |
| 33   | 8 - produce: firma canonica, ruoli e cardinalita                             | a3497f18dc46cd6af6af5caaf9ce25b3  |
| 34   | 8 - produce: identity continuity, invarianti e forbidden inference           | 90363ccc59b9cd817ff464809b0071e5  |
| 35   | 8 - produce: domande di identificazione e procedura di estrazione            | 33bf51c48bbe73a21ea78aa81bce926   |
| 36   | 8 - produce: esempio generico e worked example DermaTriage corrente          | cc6deb21f9c4c0bc45bf043617f58d0c  |
| 37   | 8 - produce: prompt LLM per l'identificazione controllata                    | 929ad404323716b6db5630c172f8b864  |
| 38   | 8 - produce: output BA atteso e discriminazioni differite                    | 29011b2ce81135be54d4b0002b9c69c8  |
| 39   | 8 - produce: rappresentazione Mermaid                                        | 7abd09f421a17323e61078e606c76cc5  |
| 40   | 8 - produce: esempio grafico Mermaid                                         | b0e191c6045d2ce9e1f7237f8a64722   |
| 41   | 9 - realize: definizione, smallest fact e criteri d'uso                      | fd686d1106dfb09f995cbd206d72f204  |
| 42   | 9 - realize: firma canonica, ruoli e cardinalita                             | 7b5f8985b2b7e03d4fa8cedf3a820e17  |
| 43   | 9 - realize: identity continuity, invarianti e forbidden inference           | 376f50ac864805c54ebbb05510dd2437  |
| 44   | 9 - realize: domande di identificazione e procedura di estrazione            | 787e4c5ce9931b03135f6f6e3237ab0c  |
| 45   | 9 - realize: esempio generico e worked example DermaTriage corrente          | 878f6fa9ebb373d1e92b53232d28d37   |
| 46   | 9 - realize: prompt LLM per l'identificazione controllata                    | 1d9565d3ea82d81047cb85703c2209f8  |
| 47   | 9 - realize: output BA atteso e discriminazioni differite                    | 541905031f0138d136150e7ae4a8c356  |
| 48   | 9 - realize: rappresentazione Mermaid                                        | 452922a9ba3eda4dd4fa6a4e3e8f262   |
| 49   | 9 - realize: esempio grafico Mermaid                                         | ea668a6b4e75d6ae2af23bdd0f9f313e  |
| 50   | 10 - constrain: definizione, smallest fact e criteri d'uso                   | 8eaeccceab767d383792c89556a2de0a5 |
| 51   | 10 - constrain: firma canonica e forme di constraintValue                    | 19e590e985946f03fa9d509114958c3b  |
| 52   | 10 - constrain: identity continuity, invarianti e forbidden inference        | c1b9d6fafaf9462c73f4489fad3664d   |
| 53   | 10 - constrain: domande di identificazione e procedura di estrazione         | ed20ede9432d24c5fee8c50f3d80f862  |
| 54   | 10 - constrain: esempio generico e worked example DermaTriage corrente       | 9866d0d6f214e0b181ad9015f5137ad9  |
| 55   | 10 - constrain: prompt LLM per l'identificazione controllata                 | 52f8b97d73181ebc63f4ffcc2419beed0 |
| 56   | 10 - constrain: output BA atteso e discriminazioni differite                 | 1e66cad562b25b4881d43e62c217878b  |
| 57   | 10 - constrain: rappresentazione Mermaid                                     | b4c3de1c60fb207da9be278f13f0263e  |
| 58   | 10 - constrain: esempio grafico Mermaid composto con transfer                | 9ac0774d88929da648825995c1076378  |
| 59   | 11 - condition: definizione, smallest fact e criteri d'uso                   | ff13530c402da52d442b404aa8b443d7  |
| 60   | 11 - condition: forma canonica, scope locale e cardinalita                   | d5d06eb1db7d0bce9416648ab93e2c02  |
| 61   | 11 - condition: identity boundary, invarianti e forbidden inference          | a6de99c516eda39f3ac29f1a6246bb90  |
| 62   | 11 - condition: domande di identificazione e procedura di estrazione         | ec89bcdef246536ca693660807fae78d  |
| 63   | 11 - condition: esempio generico e worked example DermaTriage corrente       | 5693f684c29fcc1a655605d77b85877d  |
| 64   | 11 - condition: prompt LLM per l'identificazione controllata                 | 3cef93f3dcd6726629e1dd667c5c0f775 |
| 65   | 11 - condition: output BA atteso e discriminazioni                           | 7956aba2ac06ebcc3c7973d91b70469a  |
| 66   | 11 - condition: rappresentazione Mermaid e semantic determinism              | 4d65df60cd00baef5ce9044dbbc85f99  |
| 67   | 11 - condition: esempio grafico Mermaid composto con produce                 | ecda2667c8851cd4a8a88c0b5ab7a817  |
| 68   | 12 - decisionRule: definizione, smallest fact e criteri d'uso                | 838b86747edfaf9255eba656359ba686  |
| 69   | 12 - decisionRule: forma canonica candidate e cardinalita                    | ae76776a0123bf292f75f6e6c8e85873  |
| 70   | 12 - decisionRule: identity boundary, invarianti e forbidden inference       | 81fa8e48c04924374f844530857a7961  |
| 71   | 12 - decisionRule: domande di identificazione e procedura di estrazione      | a0be33fafc8818cbe8aa9a228800ae06  |
| 72   | 12 - decisionRule: esempio generico e pressure case DermaTriage              | 78df03da071146ea0784e3cb663f8e1   |
| 73   | 12 - decisionRule: prompt LLM per l'identificazione controllata              | e71b51c4b33009106e6b6a6244365314  |
| 74   | 12 - decisionRule: output candidate e discriminazioni                        | 31a35f3c5c6d7bc55d535686970e2869  |
| 75   | 12 - decisionRule: rappresentazione Mermaid e visibility profile             | f248e4c48942111fde0c866c423d5e24  |
| 76   | 12 - decisionRule: esempio grafico DermaTriage candidate                     | ba2f8053b6612bc2a52396c2fd3ec949  |
| 77   | Appendice temporanea - scopo, lineage e confine del rebuild                  | 3915eae02f714ac47e0df4764b2836f   |
| 78   | Appendice temporanea - corpus e stati epistemici                             | 5475a02c3223c3fac62a7eebe62f6be2  |
| 79   | Appendice temporanea - Construct Description Contract e principio espositivo | e1b1becfc138e20474529fae47d6ea05  |
| 80   | Appendice temporanea - CDC direzione della spiegazione e revisione           | 435e9ecd9db23d230df9cc32766e10610 |
| 81   | Appendice temporanea - CDC campi 1-10                                        | 9d02d27cc2ad8bb4e1b30d441667e854  |
| 82   | Appendice temporanea - CDC campi 11-20                                       | 90b6659e5a0105e0cd7268d3f76137f6  |
| 83   | Appendice temporanea - roadmap corrente del rebuild R7                       | 4076e2f7a00855a7ac650ff3ac7ab044  |
| 84   | Indice di integrita' delle pagine                                            | 080f4ff9e7080b3504b21842c5a514db  |
| 85   | Appendice temporanea - laboratorio prove grafiche Mermaid da cancellare      | 13cdd31df41e1d75e405e6ba020261fa  |

La pagina successiva resta il laboratorio grafico temporaneo da cancellare; le altre appendici di rebuild restano deliberate fino al consolidamento.

## PROVA GRAFICA TEMPORANEA - DA CANCELLARE

### Scopo della pagina

Questa pagina e' un laboratorio grafico temporaneo e non stabilisce da sola le projection definitive. Deve essere eliminata quando le convenzioni saranno stabilizzate. La prova corrente conserva la vista composta produce + transfer nella quale ClinicalDescription resta un dato passivo: e' result della produzione e content del transfer, ma non genera frecce verso la destination. Le prove di operatori successivi possono essere aggiunte qui senza trasformare questa appendice in metodologia normativa.

### Prova A - content visivamente tra source e transfer

La disposizione cercata mette ClinicalDescription fisicamente fra source e transfer. Il percorso attivo resta source -> transfer -> destination; la linea tratteggiata content e' una associazione di role e non un flusso.

flowchart LR

```
S["ClinicalDescriptionGeneration<br/>actor / source"]
C@{ shape: doc, label: "ClinicalDescription<br/>result / content" }
T(("transfer"))
D["HistoricalCaseRetrieval<br/>destination"]
B["ClinicalDescriptionTransfer<br/>behavior"]
```

```
S -->|"produce"| C
S ==> T ==> D
C -.-|"content"| T
B -.-|"behavior"| T
```

```
classDef endpoint fill:#EEF4F9,stroke:#275D86,color:#17324D
classDef content fill:#FFFFFF,stroke:#275D86,color:#17324D
classDef transfer fill:#DCEAF5,stroke:#275D86,color:#17324D
classDef behavior fill:#FFF4E3,stroke:#A96A16,color:#17324D
class S,D endpoint
class C content
class T transfer
class B behavior
```

